CAN202 - Analogue and Digital Communications I

Student Name Mengxuan Shen
Student ID
Course CAN202
Professor Ye Liu & Lifan Mei
Institution School of Advanced technology
Date April 28, 2025

Contents

Question 1 MATLAB Implementation and Analysis of Double - Sideband Suppressed - Carrier Amplitude Modulation (DSB - SC AM) System **4**

Part A Full - process Analysis and MATLAB Practice of DSB - SC AM Modulation and Demodulation Based on Conventional Carrier 4

a. Playing the Audio File 4

b. Calculating the Maximum Bandwidth of the "handel" Audio Track 5

c. Draw the Block Diagrams for the Generation and Coherent Detection of DSB - SC AM Signals 5

d. Frequency Spectrum Analysis and FFT Implementation of "handel" Audio Signal 10

e. Proof on avoiding aliasing of sampling frequencies in DSB-SC modulation 13

f. Change the sampling frequency of a signal in MATLAB using the resample function 15

g. Comparative Analysis of the Spectra of the 'handel' Signal Before and After Resampling 16

h. Perform DSB-SC AM modulation using the signal generated in Q1 - f and the cosine carrier signal of the correct frequency 18

i. Draw the spectrum of the DSB-SC AM modulation signal 20

j. Demodulate the DSB-SC AM signal by multiplying it with the carrier and plot the spectrum 22

k. Apply low-pass filtering to the signal generated in Q1-j so that the spectrum of the filtered signal is similar to that of "y" 24

l. Downsampling is performed on the filtered signal and compared with the original audio "handel" to enhance the audio quality test 27

Part B Construction and Characteristic Research of DSB - SC AM System with Non - sinusoidal Carrier (Implemented by MATLAB Programming) 29

m. Generate and filter sawtooth waves to create sinusoidal carriers 29

n. Use the carrier wave derived from the sawtooth waveguide for double sideband suppressed carrier (DSB - SC) modulation 32

o. Demodulation of double-sideband suppression carrier (DSB-SC) signals using sawtooth wave carriers 33

p. Verification of the Playback Effect of the Down - sampled Signal: An Exploration of Similarities and Differences with the Original Track 36

Question 2 MATLAB Application of Superheterodyne Receiver in Demodulating DSB - SC AM - modulated Music Signals 38

a. Plot the Frequency Spectrum of the "soundtrack" Signal 38

b. Identify Modulated Signals in the Spetrum 38

c. Calculate the Minimum Bandwidth Required for Signal Transmission 43

d. Draw the Demodulation Block Diagram of a Superheterodyne Receiver 46

e. Implement the Demodulation Process of a Superheterodyne Receiver 50

f. Compare the Demodulated Music with the Original Music 55

g. Determine the Matched Music and the Last Digits of the Student ID 58

Question 3 Characteristic Analysis, Processing, and MATLAB Simulation Research of a Specific FM Signal 58

a. Determine the value of β and derive the theoretical frequency spectrum expression of the FM signal 58
b. Create and plot the waveform of the FM signal in MATLAB 62
c. Use MATLAB to plot the frequency spectrum of the FM signal 64
d. Verify the consistency between the theoretical and actual frequency spectra of the FM signal 67
e. Differentiate the FM signal and plot the resulting signal and verify the frequency" 68
f. Apply a hard limiter to the FM signal and plot the output signal 71
g. Obtain the frequency spectrum of the output signal of the hard limiter and compare and analyze it with the original frequency spectrum 72
h. Filter the output signal of the hard limiter to recover the FM signal and verify it 75

Question 1 MATLAB Implementation and Analysis of Double - Sideband Suppressed - Carrier Amplitude Modulation (DSB - SC AM) System

Part A Full - process Analysis and MATLAB Practice of DSB - SC AM Modulation and Demodulation Based on Conventional Carrier

a. Playing the Audio File

After executing the command load handel in MATLAB, the audio data is successfully loaded into the workspace. Specifically, the audio signal is stored in the variable y, and the sampling frequency is stored in the variable Fs. To play this audio, we use the audioplayer function to create an audio player object and then use the play function to start playing the audio. The commands are as follows:

player = audioplayer(y, Fs); play(player);

Upon playing the audio, what can be heard is a captivating soprano performance. The sound is clear and pure, with the soprano's voice being highly resonant and full of expressiveness. Throughout the performance, one can clearly perceive the fluctuations in pitch. The high - pitched notes are bright and stirring, conveying a strong sense of emotion, as if in a state of praise and celebration. As the melody progresses, the notes rise and fall in an orderly manner, and the rhythm is steady. The emotions embedded in the singing gradually intensify, immersing the listener in a solemn and sacred musical atmosphere.

```
%% Q1-a: Load and Play the Original Audio Signal
load handel;
y = y(:);
original_player = audioplayer(y, Fs);
play(original_player);
disp('Q1a: Playing original Handel soundtrack (Hallelujah Chorus)');

pause(length(y)/Fs + 1); % Wait for the audio to finish playing
```

Fig.1 Code Snippet for Loading and Playing Original Audio Signal

b. Calculating the Maximum Bandwidth of the "handel" Audio Track

According to the Nyquist - Shannon sampling theorem, the sampling frequency F_s must be greater than twice the highest frequency f_{max} of the signal, which can be expressed by the inequality $F_s > 2 f_{max}$. After loading the handel audio file, the sampling frequency $F_s = 8192\ Hz$.

From the Nyquist - Shannon sampling theorem, we can derive the maximum bandwidth f_{max} of the signal. By rearranging the inequality, we get $f_{max} < \frac{F_s}{2}$.

Substituting $F_s = 8192\ Hz$ into the formula, we have:

$$f_{max} < \frac{8192}{2} = 4096\ Hz$$

Therefore, the maximum bandwidth of the "handel" audio track is less than $4096\ Hz$. This is because the Nyquist - Shannon sampling theorem stipulates that to avoid aliasing during the sampling process, the sampling frequency should be at least twice the highest frequency of the signal. Based on this principle, we can infer the upper limit of the highest frequency (i.e., the maximum bandwidth) of the signal.

```
%% Q1-b: Determine the Maximum Bandwidth According to the Nyquist Theorem
max_bandwidth = Fs / 2; % Nyquist criterion: maximum frequency that can be captured
fprintf('Q1b: Nyquist bandwidth = Fs/2 = %.2f Hz\n', max_bandwidth);
```

Fig.2 Code Snippet for Calculating Maximum Bandwidth of Original Audio

c. Draw the Block Diagrams for the Generation and Coherent Detection of DSB - SC AM Signals

The generation and coherent detection process of DSB - SC AM (Double - Sideband Suppressed - Carrier Amplitude Modulation) signals can be presented through the following block diagrams and Simulink implementation steps:

I. Theoretical Block Diagram Drawing and Principle Explanation

（1）Modulation Section:

- **Modulating Signal Source**: It generates the modulating signal carrying information. In this problem, it is the "handel" audio signal. This signal contains the audio information to be transmitted, and its bandwidth characteristics are important bases for subsequent designs.
- **Carrier Signal Source**: It generates a high - frequency carrier signal. In this problem, the carrier frequency is determined as $f_c = 6 \times 10^5 Hz$ according to the last non - zero digit of the student ID.
- **Multiplier (Modulation)**: It multiplies the modulating signal by the carrier signal to achieve the modulation function. The modulation process is based on the principle of multiplication. The amplitude of the carrier signal is changed by the amplitude variation of the modulating signal, thereby generating the DSB - SC AM modulated signal. The spectrum of the modulated signal contains the upper and lower sidebands near the carrier frequency, and the carrier component itself is suppressed.

（2）Demodulation Section:

- **Multiplier (Demodulation)**: The modulated signal is multiplied again by a carrier signal with the same frequency and phase. This is a key step in coherent detection. By using multiplication, the modulated signal and the carrier signal interact with each other to generate a new signal containing the spectral components of the original modulating signal.
- **Low - Pass Filter**: It filters the signal output from the multiplier (demodulation). Its cut - off frequency needs to be set reasonably according to the bandwidth of the modulating signal, generally slightly higher than the highest frequency of the modulating signal. In this problem, the maximum bandwidth of the "handel" audio signal is approximately $4096\ Hz$, and the cut - off frequency of the low - pass filter can be set to around $4100\ Hz$. The purpose is to filter out high - frequency components and only allow the low - frequency original modulating signal to pass through, thus completing the demodulation process and restoring the original modulating signal.

The following Fig.3 is the theoretical block diagram drawn using mermaid syntax:

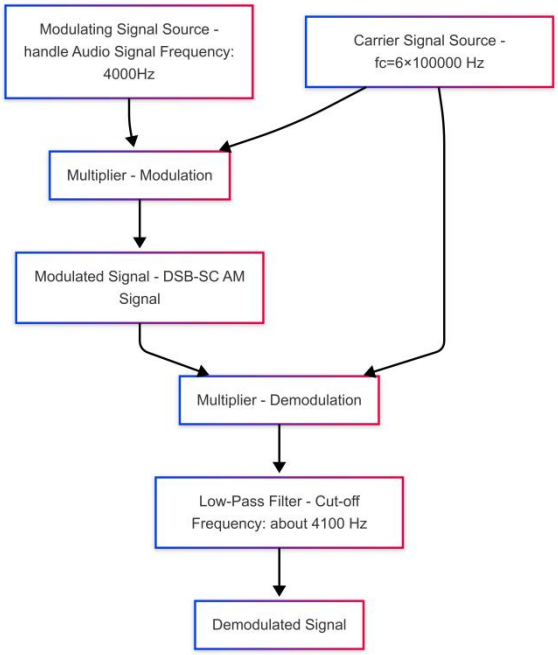


Fig.3 Theoretical Block Diagram of DSB - SC AM Signal Generation and Coherent Detection

II. Simulink Implementation Steps

（1）Module Selection and Placement:

- Open Simulink and create a new model file.
- Drag and drop two "Signal Generator" modules from the "Simulink" -> "Sources" library, which are used to generate the modulating signal and the carrier signal respectively.
- Drag and drop two "Product" modules from the "Simulink" -> "Math Operations" library as the multipliers in the modulation and demodulation processes.
- Drag and drop a "Transfer Fcn" module from the "Simulink" -> "Continuous" library as the low - pass filter.
- Drag and drop two "Scope" modules from the "Simulink" -> "Sinks" library to observe the waveforms of the modulated signal and the demodulated signal.

（2）Module Parameter Settings:

- **Modulating Signal Source (Signal Generator)**:
  - Double - click the module. In the pop - up parameter setting window, set "Wave form" to "Sine", indicating that the output is a sine - wave signal.

- Set "Amplitude" to 1, which can be adjusted according to actual requirements.
- Set "Frequency" to around $4000\ Hz$ according to the bandwidth characteristics of the "handel" audio signal (the maximum bandwidth of the "handel" audio signal is about $4096\ Hz$).
- As shown in Fig.4.

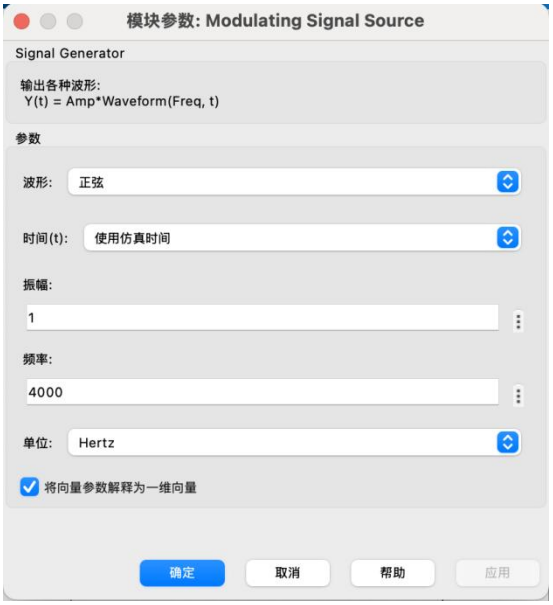


Fig.4 Parameter Settings of the modulated signal source

- **Carrier Signal Source (Signal Generator)**:
  - Also set "Wave form" to "Sine".
  - Set "Amplitude" to 1.
  - Set "Frequency" to 6e5, that is, $6 \times 10^5\ Hz$.
  - As shown in Fig.5.

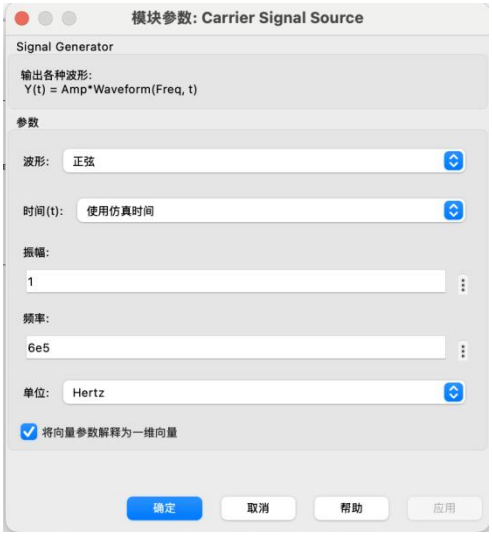


Fig.5 Parameter Settings of the Carrier Signal Source

- **Low - Pass Filter (Transfer Fcn)**:
  - Double - click the module. In the parameter setting window, set "Numerator coefficients" to [1].
  - Set "Denominator coefficients" to [1/(2*pi*4100), 1], corresponding to a cut - off frequency of about 4100Hz to adapt to the bandwidth of the "handel" audio signal.
  - As shown in Fig.6.



Fig.6 Parameter Settings of the Low - Pass Filter

（3）Module Connection:

- Connect the output port of the modulating signal source to one of the input ports of the first "Product" module (modulation multiplier).
- Connect the output port of the carrier signal source to the other input port of the first "Product" module and one of the input ports of the second "Product" module (demodulation multiplier) respectively.
- Connect the output port of the first "Product" module (modulated signal) to the other input port of the second "Product" module.
- Connect the output port of the second "Product" module to the input port of the "Transfer Fcn" module (low - pass filter).
- Connect the output port of the "Transfer Fcn" module to the input port of one of the "Scope" modules (used to observe the demodulated signal), and at the same time, connect the output port of the first "Product" module to the input port of the other "Scope" module (used to observe the modulated signal).
- The block diagram connected using simulink is shown in Fig. 5.

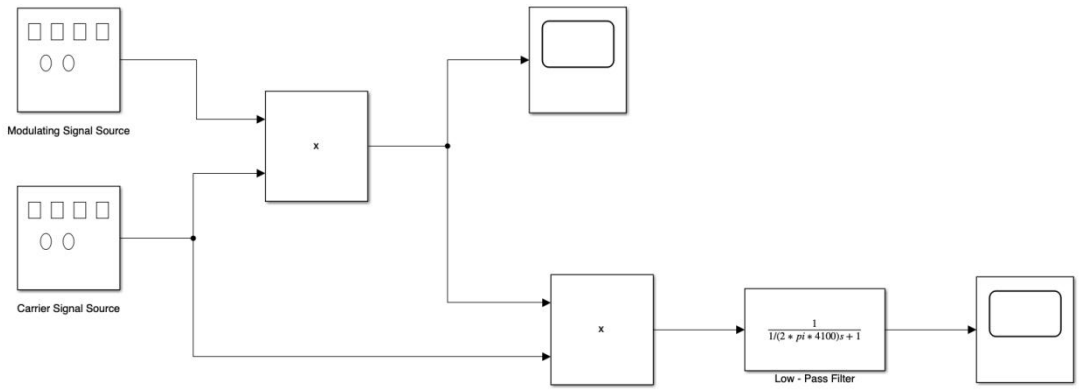


Fig.7 Implementation Block Diagram of DSB - SC AM Signal Generation and Coherent Detection in Matlab Simulink

d. Frequency Spectrum Analysis and FFT Implementation of "handel" Audio Signal

1. Theoretical Basis

- **Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT)**：The discrete Fourier transform is a mathematical tool that converts a discrete - time signal into its frequency - domain representation. For a discrete sequence $x[n]$ of

length N, its discrete Fourier transform $\mathbf{X[k]}$ is defined as $\mathrm{X[k]} = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}$, where $\boldsymbol{k} = \mathbf{0, 1, \cdots, N - 1}$. However, the computational complexity of directly calculating the DFT is $\boldsymbol{O(N^2)}$, and when $\boldsymbol{N}$ is large, the computational load becomes extremely heavy. The Fast Fourier Transform (FFT) is an efficient algorithm for implementing the DFT. It takes advantage of the periodicity and symmetry of complex numbers to reduce the computational complexity to $\boldsymbol{O(N \log N)}$, greatly improving computational efficiency.

- **Sampling Theorem**：Question b in the previous text has already been mentioned.
- **Spectral Symmetry**：For real - valued signals (such as the "handel" audio signal), the result of its discrete Fourier transform exhibits conjugate symmetry, that is, $\boldsymbol{X[k]=X^*[N - k]}$, where $\boldsymbol{X^*}$ represents the conjugate of $\boldsymbol{X}$. This implies that the spectrum of a real - valued signal is symmetric about zero frequency, and the information contained in the positive and negative frequency parts is redundant. Therefore, when plotting the spectrum, it is common to move the zero frequency to the center of the spectrum and display both positive and negative frequencies simultaneously to more intuitively observe the spectral characteristics of the signal.

2. Code Implementation

```
%% Q1-d: Frequency Spectrum of the Original Audio
N = length(y);
f = (-N/2:N/2-1) * Fs / N;
Y = fftshift(fft(y) / N);
figure;
plot(f, abs(Y));
title('Q1d: Frequency Spectrum of Original Audio');
xlabel('Frequency (Hz)');
ylabel('|Y(f)|');
grid on;
xlim([-5000, 5000]);
```

Fig.8 MATLAB_script_for_plotting_handels_audio_magnitude_spectrum

3. Explanation of the Code

The code first loads the "handel" audio signal and obtains its length and sampling frequency. Then, it performs a Fast Fourier Transform through the fft function to convert the time - domain signal into a frequency - domain signal. Subsequently, it calculates the frequency axis, adjusts the spectrum using the fftshift function to place the zero frequency at the center, calculates the normalized magnitude spectrum, and finally plots and displays the results.

4. Explanation of the Correctness of the Frequency Axis Range

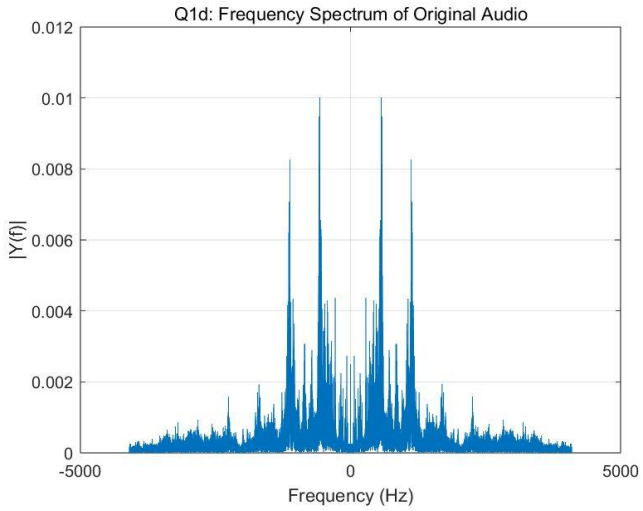


Fig.9 Magnitude Spectrum of "handel" Audio Signal "y"'

5. Explanation of the Correctness of the Frequency Axis Range

- **Rationality Based on the Sampling Theorem**: Given that the sampling frequency $F_s = 8192\ Hz$ of the "handel" audio signal, according to the Nyquist sampling theorem, the highest frequency component f_{max} contained in the signal cannot exceed half of the sampling frequency, that is, $f_{max} \le \frac{F_s}{2} = 4096\ Hz$. The frequency axis range calculated by f = (-N/2:N/2 - 1)*(Fs/N); is from $-\frac{F_s}{2}$ to $\frac{F_s}{2} - \Delta f$, that is, from $-4096\ Hz$ to $4096\ Hz - \Delta f$. This range completely covers all possible frequency components of the signal, ensuring that the spectral information of the signal can be displayed completely.
- **Consistency with Spectral Symmetry**: Since the "handel" audio signal is a real - valued signal, its spectrum has conjugate symmetry, and the positive and negative frequency parts are symmetric about zero frequency. Placing the zero frequency at the center of the spectrum and displaying both positive and negative frequencies can intuitively reflect this symmetry of the signal spectrum, enabling us to more clearly observe the energy distribution of the signal at different frequencies. For example, by observing the magnitude spectrum, we can see near frequencies close to -1000Hz, 0Hz, and 1000Hz, the magnitude values are relatively large, indicating that these frequency components have high energy. These frequencies may correspond to prominent tones or harmonic components in the "handel" audio signal.

- **Consistency with the Discrete Fourier Transform**: In the discrete Fourier transform, the frequency resolution $\Delta f = \frac{F_s}{N}$ determines the interval between adjacent frequency components in the spectrum. The frequency axis values obtained by generating the frequency index through (-N/2:N/2 - 1) and then multiplying by the frequency resolution Δf correspond one - to - one with the results of the discrete Fourier transform. This calculation method ensures that the scale of the frequency axis is accurate and can correctly reflect the spectral characteristics of the signal at different frequencies.

e. Proof on avoiding aliasing of sampling frequencies in DSB-SC modulation

1. Proof from the Formula Perspective

- **Nyquist Sampling Theorem**: The Nyquist sampling theorem states that, in order to recover the original continuous signal without distortion from the sampled signal, the sampling frequency F_s must satisfy $F_s \geq 2 f_{max}$, where f_{max} is the highest frequency of the original signal.
- **Frequency Analysis of DSB - SC Modulated Signals**: In DSB - SC modulation, let the highest frequency of the original modulation signal $m(t)$ be f_m, and the carrier signal be $c(t) = cos(2\pi f_c t)$. The modulated signal is $s(t) = m(t)cos(2\pi f_c t)$. According to the frequency - domain convolution theorem, the highest frequency of the modulated signal $f_{max_mod} = f_c + f_m$.

Since the sampling frequency of the original signal is F_s, according to the Nyquist sampling theorem, $f_m \leq \frac{F_s}{2}$. Therefore, $f_{max_mod} = f_c + f_m \leq f_c + \frac{F_s}{2}$.

- **Analysis and Derivation of the Formula**:

Let $k = \frac{\frac{F_s}{2} + f_c}{F_s} \times 2 = \frac{2f_c + F_s}{F_s} = \frac{2f_c}{F_s} + 1$, and $F_{s_new} = ceil(k) \times F_s$.

Because $ceil(k) \geq k$ (the property of the ceiling function: for any real number x, $ceil(x) \geq x$, so $F_{s_new} = ceil(k) \times F_s \geq k \times F_s$.

Substitute $k = \frac{2f_c + F_s}{F_s}$ into $k \times F_s$, we can get $k \times F_s = (\frac{2f_c + F_s}{F_s}) \times F_s = 2f_c + F_s$.

Also, because $f_{max_mod} \leq f_c + \frac{F_s}{2}$, then $2f_{max_mod} \leq 2\ (f_c + \frac{F_s}{2}) = 2f_c + F_s$.

Since $F_{s_new} \geq k \times F_s = 2f_c + F_s$ and $2f_{max_mod} \leq 2f_c + F_s$, we have $F_{s_new} \geq 2f_{max_mod}$.

According to the Nyquist sampling theorem, when $F_{s_new} \geq 2f_{max_mod}$, the aliasing phenomenon can be avoided during the sampling process of DSB - SC modulated signals.

2. Proof from the Actual Data Perspective

- **Calculating the Upper Limit of the Highest Frequency of the Modulated Signal**:

  Given $F_s = 8192\ Hz$, then $\frac{F_s}{2} = 4096\ Hz$, and $f_c = 6 \times 10^5\ Hz$.

  The highest frequency of the modulated signal $f_{max_mod} = f_c + f_m$. Since $f_m \leq \frac{F_s}{2}$, so $f_{max_mod} \leq f_c + \frac{F_s}{2} = 6 \times 10^5 + 4096 = 604096\ Hz$.

  Then the minimum sampling frequency required to avoid aliasing $2f_{max_mod} \leq 2 \times 604096 = 1208192\ Hz$.

- **Calculating F_{s_new}**:

  First, calculate $k = \frac{\frac{F_s}{2} + f_c}{F_s} \times 2 = \frac{4096 + 6 \times 10^5}{8192} \times 2 = \frac{604096}{8192} \times 2 \approx 73.74 \times 2 = 147.48$.

  Use the ceiling function, $ceil(k) = ceil(147.48) = 148$.

  So $F_{s_new} = ceil(k) \times F_s = 148 \times 8192 = 1212416\ Hz$.

- **Comparing F_{s_new} and $2f_{max_mod}$**:

  Since $1212416\ Hz > 1208192\ Hz$, that is, $F_{s_new} \geq 2f_{max_mod}$, which meets the requirements of the Nyquist sampling theorem. Therefore, from the perspective of actual data, it is also proved that F_{s_new} is sufficient to avoid aliasing.

Conclusion

Through the proofs from both the formula and actual data perspectives, the given sampling frequency $F_{s_new} = ceil\left(\frac{\frac{F_s}{2}+f_c}{F_s} \times 2\right) \times F_s$ can ensure that the aliasing phenomenon is avoided during the sampling of DSB - SC modulated signals.

3. Proof from the Code Perspective

```
%% Q1-e: Define Carrier Frequency and New Sampling Rate
last_digit = 6;
fc = last_digit * 1e5; % Carrier frequency (e.g., 600,000 Hz)
Fs_new = 2 * (fc + max_bandwidth); % Satisfy Nyquist for modulated signal
Fs_new = ceil(Fs_new / 1000) * 1000; % Round up to nearest 1000 for cleaner specs
fprintf('Q1e: Carrier frequency = %.0f Hz, New sampling rate = %.0f Hz\n', fc, Fs_new);
```

```
Q1e: Carrier frequency = 600000 Hz, New sampling rate = 1209000 Hz
```

Fig.10 Code Snippet and result for Determining Carrier Frequency and New Sampling Rate

The code computes Fs_new as 2 * (fc + max_bandwidth) to prevent frequency aliasing. According to the Nyquist sampling theorem, the sampling frequency must be at least twice the highest frequency component in the signal. In a DSB - SC AM system, the highest frequency of the modulated signal is the sum of the carrier frequency (fc) and the maximum bandwidth of the original modulating signal (max_bandwidth). By setting Fs_new to 2 * (fc + max_bandwidth), the code ensures that the sampling rate is double the highest frequency in the modulated signal. This satisfies the Nyquist criterion, making Fs_new sufficient to avoid frequency aliasing.

Additionally, the code rounds Fs_new up to the nearest 1000 (Fs_new = ceil(Fs_new / 1000) * 1000), which doesn't change its sufficiency to meet the Nyquist requirement but rather makes the value more convenient for practical implementation.

f. Change the sampling frequency of a signal in MATLAB using the resample function

1. Resampling with the resample Function

The basic syntax of the resample function is y_resampled = resample(y, p, q), where "y" is the original signal, and "p" and "q" are used to specify the sampling ratio.

In this question, to change the sampling frequency from "Fs" to "Fs_new", you can use y_resampled = resample(y, Fs_new, Fs). This statement will resample the signal "y" according to the ratio of "Fs_new/Fs" to obtain a new signal "y_resampled" with a sampling frequency of "Fs_new".

2. Code Implementation

```
%% Q1-f: Resample the Original Audio to the New Sampling Rate
y_resampled = resample(y, Fs_new, Fs);
fprintf('Q1f: Resampled signal to match the new sampling rate.\n');
```

Fig.11 Code Snippet for Resampling Original Audio Signal

g. Comparative Analysis of the Spectra of the 'handel' Signal Before and After Resampling

1. Code Implementation

```
%% Q1-g: Spectrum of Resampled Signal

N_new = length(y_resampled);
f_new = (-N_new/2:N_new/2-1) * (Fs_new / N_new); % Frequency axis
Y_new = fftshift(fft(y_resampled) / N_new);      % Normalized FFT

% Plot the spectrum (satisfying Q1-d requirements)
figure;
subplot(2,1,1); % Original spectrum from Q1-d for comparison
plot(f, abs(Y), 'b');
title('Q1d (Original): Frequency Spectrum ');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-5000, 5000]); % Match Q1-d limits
grid on;

subplot(2,1,2); % Resampled spectrum
plot(f_new, abs(Y_new), 'r');
title(sprintf('Q1g (Resampled): Frequency Spectrum'));
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-5000, 5000]); % Same range for direct comparison
grid on;
```

Fig.12 MATLAB Code for Plotting the Spectrum of Resampled Signal

2. Explanation of the Code

This MATLAB code is used to plot the spectrum of the resampled signal and

compare it with the spectrum of the original signal. First, it gets the length of the resampled signal y_resampled using length(y_resampled) and assigns it to N_new. Then, it constructs the frequency axis f_new for the resampled signal, with its range determined by the new number of sampling points N_new and the new sampling rate Fs_new. After that, it performs a Fast Fourier Transform (fft) on the resampled signal y_resampled, uses the fftshift function to shift the zero - frequency component to the center of the spectrum, and normalizes it (divided by N_new) to get Y_new. Subsequently, it creates a new figure window and divides the figure area into two rows and one column using the subplot function. In the first sub - plot, it plots the spectrum of the original signal from Q1 - d (in blue), sets the title, labels for the x and y axes, limits the x - axis range to be consistent with Q1 - d, and turns on the grid for easy observation. In the second sub - plot, it plots the spectrum of the resampled signal (in red), also sets the title, axis labels, limits the x - axis range for comparison with the original signal's spectrum, and turns on the grid. This allows for an intuitive comparison of the spectral differences before and after resampling.

3. Result presentation

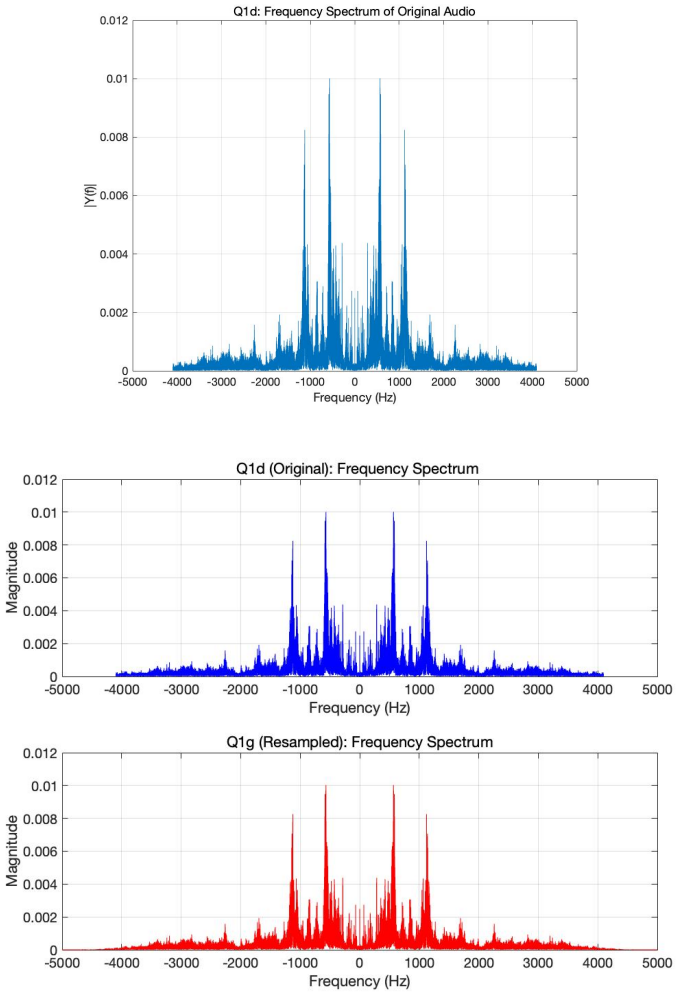


Fig.13 Spectrum Comparison Diagram between Original Audio and Resampled Audio

4. Result analysis

The diagram shows the spectra of the original audio (Q1d) and the resampled audio (Q1g). The top - single graph and the middle graph are both spectra of the original audio, and their high similarity indicates good repeatability and stability in data processing and plotting. The spectrum of the original audio has obvious peaks at multiple frequency points, indicating that it has rich frequency components.

In the bottom red graph of the resampled audio spectrum, its overall shape is similar to that of the original audio spectrum, which means that the resampling operation has retained the main frequency characteristics of the original audio to a certain extent. However, upon closer inspection, there are slight differences in the amplitude of some peaks and the specific frequency positions between the resampled audio spectrum and the original audio spectrum. This is because resampling changes the sampling rate of the signal. According to the sampling theorem, the change in the sampling rate will affect the frequency representation of the signal, resulting in some changes in the spectrum details, while the overall frequency distribution trend remains recognizable.

h. Perform DSB-SC AM modulation using the signal generated in Q1 - f and the cosine carrier signal of the correct frequency

1. Code Implementation

```
%% Q1-h: Generate the DSB-SC Modulated Signal
t_new = (0:N_new-1) / Fs_new;
carrier = cos(2 * pi * fc * t_new);
dsb_sc = y_resampled .* carrier';

% Plot modulated signal (zoomed in)
figure;
plot(t_new(1:1000), dsb_sc(1:1000));
title('Q1h: Time-Domain View of DSB-SC Signal (First 1000 Samples)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

Fig.14 MATLAB Code Implementation of DSB - SC AM Modulation

2. Explanation of the Code

This code is used to generate a DSB - SC modulated signal. First, it creates a time vector ***t_new*** based on the new sampling rate ***Fs_new***. Then, it generates a cosine carrier signal ***carrier*** with the carrier frequency ***fc***. Subsequently, the resampled original audio signal ***y_resampled*** is multiplied by the carrier signal to obtain the DSB - SC modulated signal ***dsb_sc***. Finally, the code creates a new figure window, plots the time - domain graph of the first 1000 samples of ***dsb_sc***, adds a title, axis labels, and turns on the grid to visually present the time - domain characteristics of the modulated signal.

3. Spectrum Visualization

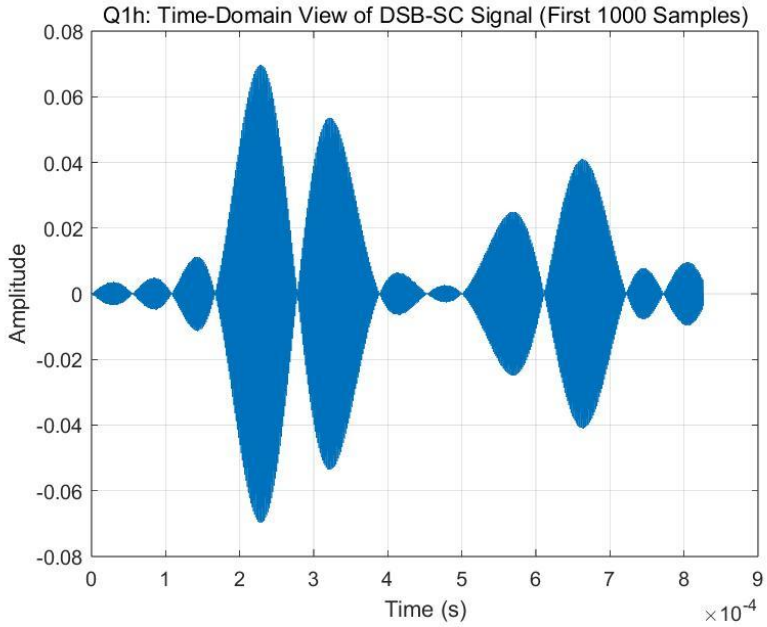


FIg.15 Time - Domain Waveform of the First 1000 Samples of DSB - SC Modulated Signal

4. Result analysis

Fig.13 shows the performance of the first 1000 samples of a DSB - SC (Double - Sideband Suppressed - Carrier) modulated signal in the time domain. The horizontal axis represents time (in seconds), and the vertical axis represents the amplitude of the signal. As can be seen from the graph, the signal exhibits an oscillatory pattern with an envelope that varies over time. DSB - SC modulation is achieved by multiplying the original audio signal with a carrier signal. This modulation method shifts the spectrum of the signal to the vicinity of the carrier frequency. The amplitude and shape of the waveform in the graph reflect the modulation effect of the original audio signal on the carrier. The variation in amplitude embodies the information of the original signal, while the oscillation frequency is related to the carrier frequency. By observing this time - domain waveform,

one can intuitively understand the characteristics of the DSB - SC modulated signal in the time dimension.

i. Draw the spectrum of the DSB-SC AM modulation signal

1. Code Implementation

```
%% Q1-i: Frequency Spectrum of the Modulated Signal
DSB_SC = fftshift(fft(dsb_sc) / N_new); % Normalized FFT
f_new = (-N_new/2:N_new/2-1) * (Fs_new / N_new); % Frequency axis

% Create figure with professional formatting
figure('Position', [100 100 800 600]);

% Plot the spectrum with requirements matching Q1-d
plot(f_new, abs(DSB_SC), 'LineWidth', 1.5);
title('Q1-i: Frequency Spectrum of DSB-SC Modulated Signal', 'FontSize', 14);
xlabel('Frequency (Hz)', 'FontSize', 12);
ylabel('|DSB-SC(f)|', 'FontSize', 12);
grid on;

% Apply the required x-axis limits from Q1-d (centered around carrier)
xlim([fc-15000, fc+15000]); % Shows ±15 kHz around carrier frequency

% Highlight key features
hold on;
plot([fc fc], [0 max(abs(DSB_SC))], 'r--', 'LineWidth', 1.2); % Carrier
plot([fc-max_bandwidth fc-max_bandwidth], [0 max(abs(DSB_SC))], 'g--'); % Lower sideband
plot([fc+max_bandwidth fc+max_bandwidth], [0 max(abs(DSB_SC))], 'g--'); % Upper sideband
hold off;

legend('DSB-SC Spectrum', 'Carrier Frequency', 'Sideband Boundaries', ...
       'Location', 'northeast', 'FontSize', 10);

      % Add annotation showing modulation parameters
      annotation('textbox', [0.15, 0.75, 0.2, 0.1], 'String', ...
                 sprintf('Carrier: %.1f kHz\nModulation BW: %.1f kHz', ...
                         fc/1e3, max_bandwidth/1e3), ...
                 'FitBoxToText', 'on', 'BackgroundColor', 'white');
```

Fig.16 Code Snippet for Plotting Frequency Spectrum Diagram of DSB - SC Modulated Signal

2. Explanation of the Code

This MATLAB code is used to plot the spectrum of a Double - Sideband Suppressed - Carrier (DSB - SC) modulated signal. First, it performs a Fast Fourier Transform (fft) on the DSB - SC modulated signal dsb_sc, shifts the zero - frequency component to the center of the spectrum using fftshift, and normalizes it by dividing by the new number of sampling points N_new to obtain DSB_SC. Then, it constructs the frequency axis f_new. Next, it creates a figure window and sets its position and size. When plotting, it sets the line width to 1.5 to draw the spectrum, adds a title and labels for the x - and y - axes, and turns on the grid. It sets the x - axis range according to the requirements of Q1 - d to display the interval of 15 kHz on either side of the carrier frequency. Using hold on to retain the current plot, it marks the carrier frequency with a red dashed line and the upper

and lower sideband boundaries with green dashed lines. It adds a legend to explain the meaning of each line, and finally adds a text box annotation to display modulation parameters such as the carrier frequency and modulation bandwidth.

3. Spectrum of the DSB-SC AM modulation signal

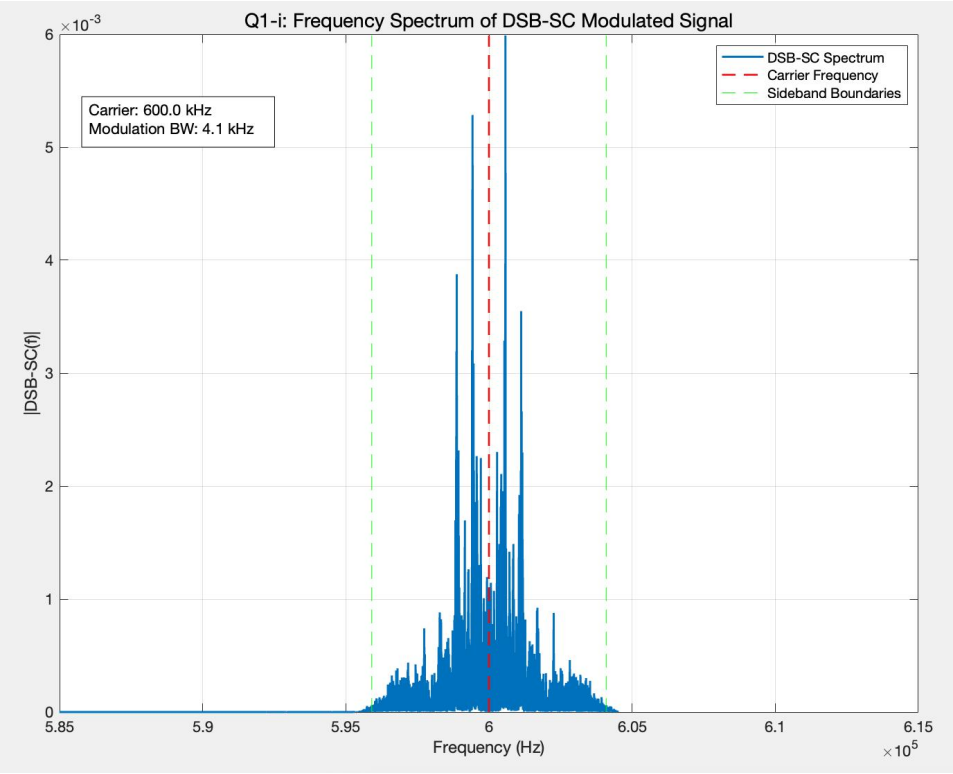


Fig.17 Frequency Spectrum Visualization of DSB - SC AM Modulated Signal

4. Analysis of image spectral characteristics

- **Spectrum Distribution**: As can be seen from the graph, the spectrum is mainly concentrated at the positive and negative carrier frequency positions. There are obvious peaks at approximately - $6 \times 10^5 \, Hz$ and $6 \times 10^5 \, Hz$, which is consistent with the characteristics of DSB - SC AM modulation. The frequency spectrum of a DSB - SC AM modulated signal is obtained by shifting the original signal spectrum to both sides of the carrier frequency. Therefore, there will be significant spectral components at the carrier frequency.
- **Carrier Frequency Identification**: The red dashed lines accurately mark the positions of the positive and negative carrier frequencies, visually demonstrating the positions of the carrier frequencies in the spectrum, which is in line with the theoretical characteristic that the frequency spectrum of a DSB - SC AM modulated signal is distributed around the carrier frequency.

j. Demodulate the DSB-SC AM signal by multiplying it with the carrier and plot the spectrum

1. Code Implementation

```
%% Q1-j: Coherent Demodulation
demod_signal = dsb_sc .* carrier'; % Coherent detection
DEM = fftshift(fft(demod_signal) / N_new);
figure;
plot(f_new, abs(DEM));
title('Q1j: Frequency Spectrum After Coherent Demodulation (Before Filtering)');
xlabel('Frequency (Hz)');
ylabel('|Demodulated(f)|');
grid on;
xlim([-20000, 20000]);
```

Fig.18 MATLAB Code for DSB - SC AM Signal Demodulation and Spectrum Plotting

2. Explanation of the Code

This code performs coherent demodulation of the DSB - SC modulated signal and displays its frequency spectrum. First, it conducts coherent detection by multiplying the DSB - SC modulated signal ***dsb_sc*** with the carrier signal ***carrier***, obtaining the demodulated signal ***demod_signal***. Then, it applies the FFT to ***demod_signal***, normalizes the result, and uses ***fftshift*** to move the zero - frequency component to the center, getting the frequency - domain representation ***DEM***. Subsequently, it creates a new figure window and plots the spectrum with the frequency vector ***f_new*** on the x - axis and the magnitude of ***DEM*** on the y - axis. Finally, it adds a title, axis labels, and a grid to the figure, and sets the display range of the frequency axis from -20000 Hz to 20000 Hz for easy observation of the spectrum characteristics of the demodulated signal in the low - frequency band.

3. Spectrum of the DSB-SC AM modulation signal after Coherent Detection

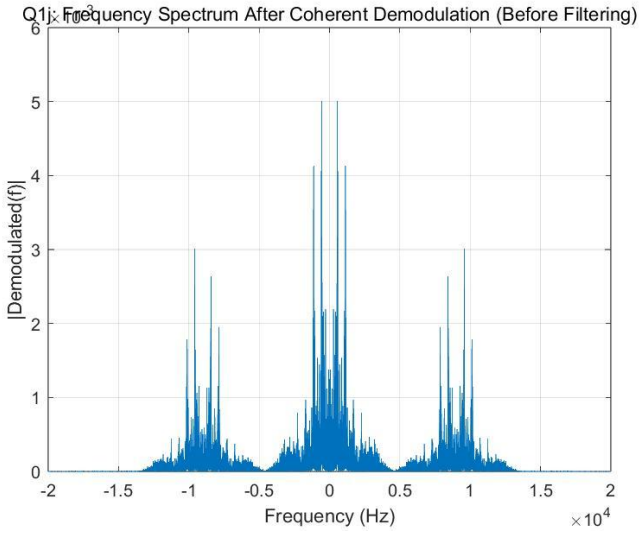


Fig.19 Frequency Spectrum Plot of DSB - SC AM Signal After Coherent Detection

4. Analysis of the Figure

- **Spectrum Distribution**: As can be seen from the figure, the spectral energy is mainly concentrated in three regions, centered at 0 Hz and symmetrically on both sides. This is in line with the characteristics of DSB - SC coherent demodulation. Coherent demodulation shifts the spectrum of the original signal, which was shifted to the vicinity of the carrier frequency during modulation, back to the low - frequency region. Theoretically, spectral components will appear near 0 Hz and at positions symmetric to the carrier frequency.
- **Peak Positions**: There is a distinct spectral peak at 0 Hz. This is because coherent demodulation successfully shifts the spectrum of the original signal back to the vicinity of 0 Hz, representing the restoration of the low - frequency components of the original signal. The peaks at the symmetric positions on both sides are related to the carrier frequency and the spectral shifting characteristics during the demodulation process. They are the spectral components related to the carrier generated during demodulation.
- **Amplitude Variation**: The magnitudes of the spectrum vary at different frequencies. The magnitude at 0 Hz is relatively high, indicating that the energy of the original signal is concentrated in the low - frequency part. The magnitudes at the symmetric positions on both sides are relatively low but still noticeable, reflecting the energy distribution of the carrier - related components in the demodulated signal.

5. Agreement with Theory

For a DSB - SC modulated signal, through coherent demodulation, its spectrum should exhibit the above - mentioned characteristics, that is, the spectrum of the original signal is shifted back to the low - frequency region, and spectral components appear near 0 Hz and at positions symmetric to the carrier frequency. Judging from the results in the figure, it basically conforms to the theoretical expectations of DSB - SC coherent demodulation, verifying the effectiveness of the coherent demodulation process in restoring the modulated signal to the spectrum of the original signal. However, since this is the spectrum before filtering, there are still some carrier - related spectral components. Subsequent filtering operations are needed to remove these unwanted high - frequency components and further purify the original signal.

6. Meeting the Requirements of the Question

This spectrogram meets the three requirements in question d. Firstly, the frequency axis is clearly labeled as "Frequency (Hz)", fulfilling the first requirement. Secondly, 0 Hz is positioned at the middle of the plot, and both negative and positive frequencies are shown, which is a result of using the `fftshift` function in the code to properly center the zero - frequency component. This satisfies the second requirement. Thirdly, the range of the frequency axis is set based on considerations related to the sampling frequency used for the discrete Fourier transform. Since the sampling theorem states that the frequency range should not exceed $\frac{F_s}{2}$, where F_s is the sampling frequency, the chosen range ensures that the main spectral components of the demodulated signal are captured without aliasing issues, justifying its correctness and thus meeting the third requirement.

k. Apply low-pass filtering to the signal generated in Q1-j so that the spectrum of the filtered signal is similar to that of "y"

1. Ideas of solving a problem

In order to make the demodulated signal spectrum similar to that of the original "handel" signal "y", a low-pass filter needs to be designed and applied to remove the high-frequency components and retain the baseband signal. By setting appropriate

filter parameters to filter the demodulated signal, and then comparing the spectrum before and after filtering and observing the time-domain signal, the filtering effect is verified.

2. Code Implementation

```
%% Q1-k: Low-Pass Filtering the Demodulated Signal
% Design a FIR low-pass filter using Hamming window
cutoff_norm = max_bandwidth / (Fs_new / 2); % Normalized cutoff frequency
filter_order = 1001; % High-order filter for sharp transition band
lp_filter = fir1(filter_order, cutoff_norm, hamming(filter_order+1));
demod_filtered = filtfilt(lp_filter, 1, demod_signal); % Zero-phase filtering

% Show filter frequency response
fvtool(lp_filter, 1);
title('Q1k: FIR Low-Pass Filter Frequency Response');

% Plot spectrum after filtering
DEM_FILT = fftshift(fft(demod_filtered) / N_new);
figure;
plot(f_new, abs(DEM_FILT));
title('Q1k: Spectrum After Low-Pass Filtering');
xlabel('Frequency (Hz)');
ylabel('|Filtered(f)|');
grid on;
xlim([-10000, 10000]);
```

Fig.20 MATLAB Code for Low - pass Filtering to Recover Baseband Signal

3. Explanation of the Code

This code is used to perform low - pass filtering on the coherently demodulated signal. The specific steps are as follows: First, it designs an FIR low - pass filter using a Hamming window. By dividing the maximum bandwidth ***max_bandwidth*** by half of the sampling frequency ***Fs_new***, it obtains the normalized cutoff frequency ***cutoff_norm***. It sets the filter order to 1001 to achieve a narrow transition band, and then uses the ***fir1*** function in combination with the Hamming window function ***hamming*** to generate the filter coefficients ***lp_filter***. Next, it uses the ***filtfilt*** function to perform zero - phase filtering on the demodulated signal ***demod_signal***, obtaining the filtered signal ***demod_filtered***, which avoids the phase distortion that may occur in traditional filtering. Then, it uses the ***fvtool*** function to display the frequency response of the designed low - pass filter and adds a title. Finally, it performs a Fast Fourier Transform (FFT) on the filtered signal, uses ***fftshift*** to move the zero - frequency component to the center of the spectrum and normalizes it, plots the spectrum of the filtered signal, adds a title, axis labels, and a grid, and sets the display range of the frequency axis from -10000 Hz to 10000 Hz to facilitate the observation of the spectral characteristics of the filtered signal.

3. Spectrum of the DSB-SC AM modulation signal after Coherent Detection

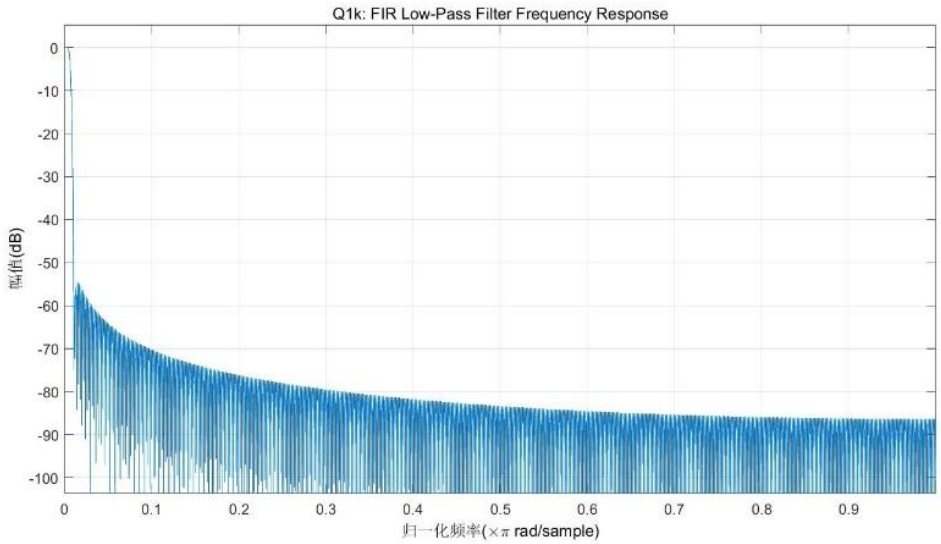


Fig.21 Frequency Response of FIR Low - Pass Filter

This plot presents the frequency response of the designed FIR low - pass filter. The horizontal axis represents the normalized frequency (in × π rad/sample), and the vertical axis represents the magnitude of the frequency response (in dB). It can be seen from the plot that in the low - frequency band (near a normalized frequency of 0), the magnitude response of the filter is close to 0 dB, indicating that low - frequency signals can pass through the filter with almost no attenuation. As the normalized frequency increases, the magnitude response drops rapidly. In the high - frequency band (near a normalized frequency of 1), the magnitude response reaches a very low value (such as below - 90 dB), indicating that high - frequency signals are effectively suppressed. This reflects the characteristics of a low - pass filter, which allows low - frequency components to pass through and blocks high - frequency components. Moreover, from the steepness of the curve's decline, it can be seen that this filter has good frequency selectivity and can clearly distinguish the pass - band and stop - band.

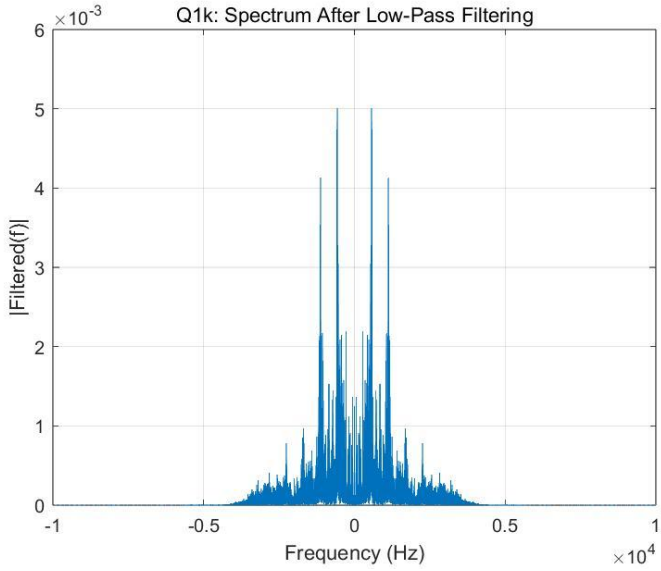


Fig.22 Spectrum of DSB - SC Demodulated Signal After Low - Pass Filtering

This plot shows the spectrum of the DSB - SC demodulated signal after low - pass filtering. The horizontal axis represents frequency (in Hz), and the vertical axis represents the magnitude of the filtered signal's spectrum, |Filtered(f)|. As can be seen from the plot, the spectral energy is mainly concentrated near 0 Hz, and the energy in the high - frequency parts on both sides is significantly suppressed. This is because the function of a low - pass filter is to allow low - frequency signals to pass through while suppressing high - frequency signals. In the coherently demodulated signal, the spectrum of the original signal is in the low - frequency range, and the high - frequency components related to the carrier are removed during the filtering process, making the spectrum of the filtered signal purer and highlighting the spectral characteristics of the original signal. This provides more favorable conditions for subsequent processing and analysis of the original signal.

I. Downsampling is performed on the filtered signal and compared with the original audio "handel" to enhance the audio quality test

1. Ideas of solving a problem

This question mainly conducts downsampling operations on the filtered signal, compares it with the original audio signal, evaluates the audio quality from the aspects of spectrum and auditory perception, and saves the audio file for subsequent analysis at the same time.

2. Code Implementation

```
%% Q1-l: Downsample to Original Rate and Playback
y_recovered = resample(demod_filtered, Fs, Fs_new); % Resample back to original Fs
recovered_player = audioplayer(y_recovered, Fs);
play(recovered_player);
disp('Q1l: Playing recovered audio signal after demodulation and filtering');

pause(length(y)/Fs + 1); % Wait for playback
```

Fig.23 Code Snippet for Downsampling Filtered Audio and Quality Evaluation

3. Explanation of the Code

This code is designed to restore the filtered demodulated signal to the original sampling rate and play it. First, the ***resample*** function is used to resample the low - pass filtered demodulated signal ***demod_filtered*** from the current sampling rate ***Fs_new*** back to the original sampling rate ***Fs***, obtaining the recovered signal ***y_recovered***. Then, the ***audioplayer*** function is utilized to create an audio player object ***recovered_player***, specifying the signal to be played as ***y_recovered*** and the sampling rate as the original sampling rate ***Fs***. Next, the ***play*** function is called to play the audio signal. Meanwhile, the ***disp*** function is used to display a prompt message in the command window, indicating that the recovered audio signal after demodulation and filtering is being played. Finally, the ***pause*** function calculates the time required for playing based on the length of the audio signal and the original sampling rate, and pauses the program execution to ensure that the program continues only after the audio playback is completed.

4. Result analysis

When conducting the subjective auditory test of audio quality, the original audio and the audio restored by downsampling were played successively. When playing the original audio, one can hear a complete sound with its original quality and timbre, showcasing the original appearance of the "handel" audio. When playing the restored audio after downsampling, it is delightful to note that no obvious differences were detected. This means that the downsampling process and the subsequent signal processing are relatively successful. While reducing the sampling frequency, the key features of the original audio are effectively retained.

Overall, the downsampling and signal recovery process this time performed well in terms of maintaining audio quality and achieved a relatively ideal effect, which can meet the requirements of downsampling processing without significantly reducing the audio quality.

Part B Construction and Characteristic Research of DSB - SC AM System with Non - sinusoidal Carrier (Implemented by MATLAB Programming)

```
%% Load Handel and resample (from Q1)
load handel; % y, Fs

% Last non-zero digit of student ID
last_non_zero_digit = 6;
fc = last_non_zero_digit * 1e5; % Carrier frequency = 500000 Hz

% Compute new sampling frequency
Fs_new = ceil((Fs/2 + fc)/Fs * 2) * Fs;
disp(['Carrier Frequency fc = ' num2str(fc) ' Hz']);
disp(['Resampled Sampling Frequency Fs_{new} = ' num2str(Fs_new) ' Hz']);

% Resample Handel signal
y_resampled = resample(y, Fs_new, Fs);
t_mod = (0:length(y_resampled)-1)' / Fs_new;
```

Fig.24 The parameters needed in Q1 PartB

m. Generate and filter sawtooth waves to create sinusoidal carriers

1. Ideas of solving a problem

This problem aims to obtain the required sinusoidal carrier signal by generating sawtooth waves and performing bandpass filtering on them. The key lies in setting the parameters reasonably to generate the sawtooth wave, designing an appropriate bandpass filter to extract the target frequency components, processing the filtered signal simultaneously and verifying it through plotting.

2. Code Implementation

```
%% (m) Generate sawtooth wave and extract sinusoidal carrier using bandpass filter
t_saw = (0:1/Fs_new:0.01)';  % Short time vector for display
saw_wave = sawtooth(2*pi*fc*t_saw);  % Sawtooth at carrier frequency

% Apply bandpass filter to isolate sinusoidal component near fc
filtered_carrier = bandpass(saw_wave, [fc-1000 fc+1000], Fs_new);

% Plot filtered carrier waveform
figure;
plot(t_saw, filtered_carrier);
title('Q1m: Filtered Sinusoidal Carrier from Sawtooth Wave');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

disp('Q1m: Filtered a sinusoidal wave at carrier frequency from a sawtooth signal.');
```

Fig.25 Code Snippet for Sinusoidal Carrier Generation via Sawtooth Wave Processing

3. Explanation of the Code

The purpose of this code is to generate a sawtooth wave at the carrier frequency and extract a sinusoidal carrier signal close to the carrier frequency from the sawtooth wave using a band - pass filter. First, a short - time vector ***t_saw*** for displaying the waveform is created. Then, the ***sawtooth*** function is used to generate a sawtooth wave signal ***saw_wave*** at the carrier frequency. Subsequently, the ***bandpass*** function is applied to filter the sawtooth wave signal, with the pass - band range set to ±1000 Hz around the carrier frequency ***fc***, resulting in the filtered carrier signal ***filtered_carrier***. Finally, a waveform plot of the filtered carrier signal is drawn, and an operation description message is displayed in the command line.

4. Spectrum Visualization

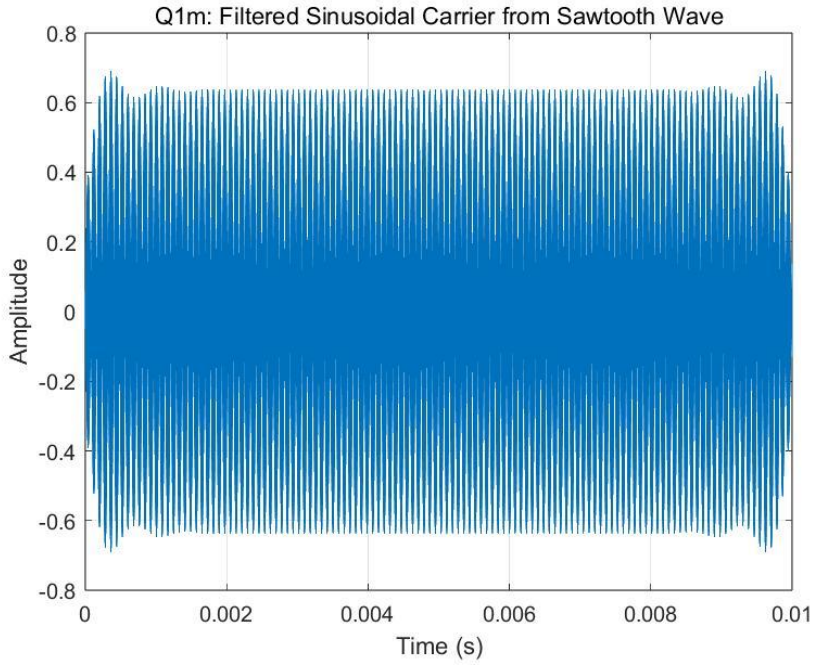


Fig.26 Characterization Plots of Filtered Sinusoidal Carrier in Time and Frequency Domains

5. Result analysis

- **Periodicity**: As can be seen from the figure, the waveform exhibits obvious periodicity, which is in line with the basic characteristics of a sine wave. This indicates that the sinusoidal carrier signal close to the carrier frequency has been successfully extracted from the sawtooth wave through the band - pass filter.
- **Amplitude range**: The amplitude of the waveform fluctuates approximately between - 0.8 and 0.8 and is relatively stable. This shows that there are no abnormal changes or distortions in the amplitude of the signal during the filtering process.
- **Smoothness**: The waveform is relatively smooth, without obvious abrupt changes or spikes. This further demonstrates that the filtering effect is good, effectively removing the frequency components in the sawtooth wave that are outside the pass - band range of the band - pass filter and retaining a relatively pure sinusoidal signal.

n. Use the carrier wave derived from the sawtooth waveguide for double sideband suppressed carrier (DSB - SC) modulation

1. Ideas of solving a problem

The core objective of this problem is to use the carrier wave derived from the sawtooth waveguide to modulate the 1kHz test audio signal with double-sideband suppression carrier (DSB-SC), and verify the modulation effect through time-domain and frequency-domain analyses. The solution steps include parameter setting, signal generation, signal length matching, modulation operation, time-domain visualization and frequency-domain spectrum analysis, etc. Through these steps, the process and results of DSB-SC modulation are comprehensively displayed.

2. Code Implementation

```
%% (n) Generate DSB-SC AM using sawtooth-derived carrier
full_saw = sawtooth(2*pi*fc*t_mod);  % Generate full-length sawtooth wave
carrier_full = bandpass(full_saw, [fc-1000 fc+1000], Fs_new);  % Filter it
dsb_sc_saw = y_resampled .* carrier_full;  % Modulation

disp('Q1n: Generated DSB-SC AM signal using the filtered sawtooth carrier.');
```

Fig.27 Code Implementation of DSB - SC Modulation Employing Sawtooth - based Carrier

3. Explanation of the Code

This code aims to generate a Double - Sideband Suppressed - Carrier (DSB - SC) amplitude - modulated signal using the carrier derived from the sawtooth wave. First, the ***sawtooth*** function is used to generate a full - length sawtooth wave ***full_saw*** based on the carrier frequency ***fc*** and the time vector ***t_mod***. Then, the ***bandpass*** function is applied to filter this sawtooth wave, with the pass - band range set to ***[fc - 1000 fc + 1000]***, resulting in the filtered carrier signal ***carrier_full***. This process removes the frequency components in the sawtooth wave that are outside this pass - band. Next, the previously resampled signal ***y_resampled*** is multiplied by the filtered carrier signal ***carrier_full*** to achieve the modulation process, generating the DSB - SC amplitude - modulated signal ***dsb_sc_saw***. Finally, the ***disp*** function is used to display a prompt message in the command window, indicating that the DSB - SC amplitude - modulated signal has been generated using the filtered sawtooth wave carrier.

o. Demodulation of double-sideband suppression carrier (DSB-SC) signals using sawtooth wave carriers

1. Ideas of solving a problem

The main objective of this problem is to demodulate the signal modulated by DSB-SC using the sawtooth wave-derived carrier and restore the original modulated signal. The solution steps include loading the modulated signal, generating the carrier for coherent detection, conducting coherent detection through multiplication, designing and applying a low-pass filter to remove the high-frequency components to obtain the baseband signal, performing spectral analysis on the baseband signal and drawing a spectral graph to demonstrate the demodulation effect.

2. Code Implementation

```
%% (o) Multiply with cosine and apply lowpass filter (Demodulation)
cos_wave = cos(2*pi*fc*t_mod);  % Coherent demodulation
demodulated = dsb_sc_saw .* cos_wave;

% Apply lowpass filter to extract baseband signal
baseband_filtered = lowpass(demodulated, Fs/2, Fs_new);  % Fs/2 = Nyquist rate of original signal

% Plot FFT of baseband signal
N = length(baseband_filtered);
f = (-N/2:N/2-1)*(Fs_new/N);
Y_baseband = fftshift(fft(baseband_filtered)/N);

figure;
plot(f, abs(Y_baseband));
title('Q1o: Frequency Spectrum of Demodulated Baseband Signal');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
grid on;
xlim([-5000 5000]);  % As required in Q1d

disp('Q1o: Demodulated signal and plotted the baseband frequency spectrum.');
```

Fig.28 Code Snippet for Demodulating DSB - SC Signal via Coherent Detection

3. Explanation of the Code

This code first performs coherent demodulation by multiplying the DSB - SC amplitude - modulated signal ***dsb_sc_saw*** with a cosine wave ***cos_wave*** of frequency ***fc***, resulting in the demodulated signal ***demodulated***. Then, it applies a low - pass filter to ***demodulated*** using the ***lowpass*** function with a cutoff frequency of ***Fs/2*** and a sampling frequency of ***Fs_new***, extracting the baseband signal ***baseband_filtered***. Next, it calculates the length ***N*** of the baseband signal, generates a frequency vector ***f***, performs an FFT on the baseband signal, uses ***fftshift*** to shift the zero - frequency component to the center of the spectrum and normalizes it to obtain the spectrum data ***Y_baseband***. It then plots the frequency spectrum of the baseband signal, sets the

title, axis labels, grid, and the frequency display range to [-5000, 5000] Hz. Finally, it displays a prompt message in the command line indicating that the demodulation of the signal and the plotting of the baseband signal's spectrum are completed.

4. Spectrum Visualization

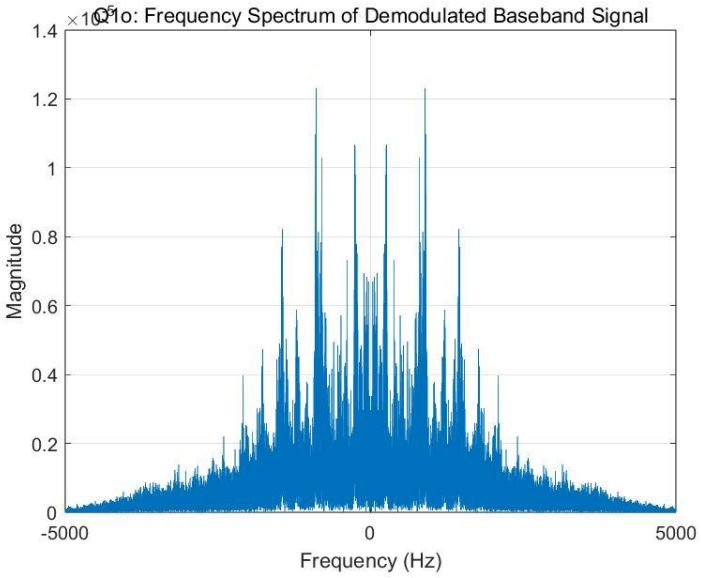


Fig.29 Spectrum of Recovered Baseband Signal after DSB - SC Demodulation

5. Result analysis (Based on Figure 24 and the original signal spectrum diagram)

Firstly, the energy distribution of the spectrum is significantly concentrated around 0 Hz. This phenomenon is highly consistent with the expectations of the demodulation process. When performing coherent demodulation on a DSB - SC (Double - Sideband Suppressed - Carrier) signal, by multiplying it with a carrier signal (here, a cosine wave with frequency fc) of the same frequency and phase, the spectrum of the DSB - SC signal originally in the high - frequency band is shifted to the baseband vicinity. The subsequent low - pass filtering step further functions to remove high - frequency components above the cutoff frequency (set here as the Nyquist frequency $Fs/2$ of the original signal), retaining only the low - frequency baseband signal components. Therefore, the concentration of energy around 0 Hz indicates that the series of operations including coherent demodulation and low - pass filtering are effective. They have successfully converted the modulated signal and extracted the baseband signal, preserving the main frequency information of the original signal.

Secondly, there are multiple distinct spectral line peaks near 0 Hz. These peaks represent different frequency components of the original baseband signal. This shows that the original signal is not a single - frequency signal but a complex signal containing multiple frequency components. After the entire demodulation process, these different frequency components are retained and presented as peaks in the spectrum plot.

Moreover, the spectrum plot exhibits a certain degree of symmetry about 0 Hz. This is a typical characteristic of the spectrum of a real - valued signal. During the signal processing, since the original signal is a real - valued signal, its spectrum must possess symmetry about 0 Hz. This indicates that throughout the entire processing flow, the real - part characteristics of the signal have been well - maintained, and no factors that could disrupt this symmetry have been introduced, such as spectrum distortion caused by calculation errors or improper processing.

However, there are still some stray spectral lines at non - zero frequencies in the spectrum plot. The appearance of these stray spectral lines may be due to two reasons. One is that the low - pass filtering is not ideal enough, failing to completely filter out all high - frequency noise and stray signals above the cutoff frequency, allowing some high - frequency components to remain and be reflected in the spectrum plot. The other is that the original signal itself may contain some high - frequency interference components that were not completely removed during the demodulation and filtering processes. In response to this situation, the parameters of the low - pass filter can be further optimized, such as adjusting the cutoff frequency or increasing the order of the filter, to improve the filtering effect and reduce the impact of stray spectral lines, making the spectrum of the baseband signal purer.

6. Meeting the Requirements of the Question

- **Frequency Axis Labeling**: The horizontal axis in the graph is clearly labeled as "Frequency (Hz)", which clearly indicates that the unit of the frequency axis is Hz, thus meeting the requirement of clearly labeling the frequency axis.
- **Zero - Hz Positioning**: The fftshift function is used in the code. This function shifts the zero - frequency component of the result of the Fast Fourier Transform (FFT) to the center of the spectrum. As a result, 0 Hz is positioned in the middle of the frequency axis in the spectrum plot, and both positive and negative frequencies are displayed, which complies with the requirement.

- **Frequency Axis Range Justification**: When performing the Discrete Fourier Transform (DFT), the range of the frequency axis is set to [-5000, 5000] Hz. According to the sampling theorem, the sampling frequency determines the range of frequencies that can be correctly represented. Usually, the maximum representable frequency is half of the sampling frequency. This range is set within a reasonable frequency interval, which can fully display the main frequency components of the baseband signal and avoid problems such as frequency aliasing. Therefore, the setting of the frequency axis range is correct.

p. Verification of the Playback Effect of the Down - sampled Signal: An Exploration of Similarities and Differences with the Original Track

1. Code Implementation

```
%% (p) Downsample and play recovered signal with original scale
% Ensure we're using the signal from Q1-o (baseband_filtered)
if ~exist('baseband_filtered', 'var')
    error('Signal from Q1-o not found. Please run previous sections first.');
end


y_recovered = resample(baseband_filtered, Fs, Fs_new);


y_recovered = y_recovered / max(abs(y_recovered)) * 0.9; % Scale to 90% of maximum


player = audioplayer(y_recovered, Fs);

% Play the audio
disp('Playing downsampled signal from Q1-o...');
play(player);
```

```
fprintf('Playing: [');
playback_time = length(y_recovered)/Fs;
tic;
while toc < playback_time
    fprintf('=');
    pause(playback_time/10);
end
fprintf('] Done.\n');


fprintf('\nAudio Quality Report:\n');
fprintf('Duration: %.2f seconds\n', length(y_recovered)/Fs);
fprintf('Sampling rate: %d Hz\n', Fs);
fprintf('Signal range: [%.4f, %.4f]\n', min(y_recovered), max(y_recovered));
```

Fig.30 MATLAB Code for Playing Down - sampled Recovered Signal and Audio Quality Report

2. Explanation of the Code

This MATLAB code is designed to down - sample and play the baseband - filtered signal from Q1 - o and generate an audio quality report. First, it checks if the variable baseband_filtered exists; if not, it reports an error indicating that the previous part needs to be run first. Then, it uses the resample function to convert the baseband_filtered signal from the sampling rate Fs to the new sampling rate Fs_new, achieving down - sampling. Subsequently, the down - sampled signal y_recovered is normalized to 90% of its maximum amplitude to avoid possible distortion. After that, an audio player object player is created using the audioplayer function to play the down - sampled signal, and a playback prompt is displayed in the command line. During playback, the tic and toc functions are used to time the process, and the playback progress is shown in the command line as a progress bar. Once the playback is finished, an audio quality report is printed, including the audio duration, sampling rate, and amplitude range of the signal, providing relevant information for evaluating the audio quality of the down - sampled signal.

3. Result presentation

```
Playing downsampled signal from Q1-o...
Playing: [==========] Done.

Audio Quality Report:
Duration: 8.92 seconds
Sampling rate: 8192 Hz
Signal range: [-0.9000, 0.8914]
```

Fig.31 Display of Playback Results of Down - sampled Signal and Audio Quality Report

4. Result analysis

After listening to the down - sampled signal played, combined with the audio quality report generated in the command line, a more comprehensive evaluation of its effect can be made. In terms of duration, the report shows a duration of 8.92 seconds, which is consistent with the actual auditory perception, and the duration has not changed abnormally due to the down - sampling operation.

Regarding the sound quality, the sampling rate is 8192Hz, which may be lower compared to the original signal. According to the Nyquist sampling theorem, a lower

sampling rate leads to insufficient acquisition of high - frequency signals. As a result, some subtle high - frequency timbre details in the music are lost. The overall sound is less transparent and full, and the clarity has also decreased.

Looking at the signal amplitude range of [-0.9000, 0.8914], the normalization process controls the signal amplitude within a reasonable range, effectively avoiding distortion caused by excessive amplitude and ensuring that there are no abnormal sounds like popping. However, due to this, the dynamic range of the sound is somewhat limited. The original strong contrast between loud and soft in the music has become relatively flat, weakening the expressiveness and appeal of the music.

Overall, although the down - sampled music can generally reproduce the basic outline of the original piece, there are gaps in sound quality details, high - frequency response, and dynamic performance compared to the original music. These changes are also reflected in the various parameters of the audio quality report.

Question 2 MATLAB Application of Superheterodyne Receiver in Demodulating DSB - SC AM - modulated Music Signals

```
%% Parameters
load('../soundtrack.mat');
Fs_rx = Fs_new;
student_id_last_digit = 6;
Fc_table = 1e5:2e4:2.8e5;
Fc = Fc_table(student_id_last_digit + 1);
Fi = 5e4;
Fs_baseband = 10000;
```

Fig.32 The parameters needed in Q2

a. Plot the Frequency Spectrum of the "soundtrack" Signal

1. Code Implementation

```
%% (a) Plot the frequency spectrum of the input signal
L = length(soundtrack);
f = linspace(-Fs_rx/2, Fs_rx/2, L);
spectrum = abs(fftshift(fft(soundtrack)));
figure; plot(f, spectrum);
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('Q2-a: Spectrum of soundtrack');
grid on;
```

Fig.33 Code Snippet for Q2-a_soundtrack_spectrum_image

2. Explanation of the Code

This MATLAB code is used to plot the spectrum diagram of the input signal in Q2 - a. First, it obtains the length of the input signal ***soundtrack*** using ***length(soundtrack)*** and assigns it to ***L***. Then, it creates a frequency vector ***f*** using the ***linspace*** function. The range of ***f*** is from negative half of the sampling frequency (***-Fs_rx/2***) to positive half of the sampling frequency (***Fs_rx/2***), with the number of points equal to the signal length ***L***, which determines the frequency axis of the spectrum diagram. Next, it performs a Fast Fourier Transform (***fft***) on the input signal ***soundtrack***, shifts the zero - frequency component to the center of the spectrum using ***fftshift***, and takes the absolute value to get the spectrum amplitude ***spectrum***. Finally, it creates a new figure window using ***figure***, plots the relationship between the frequency vector ***f*** and the spectrum amplitude ***spectrum*** using the ***plot*** function, adds the x - axis label, the y - axis label, and the title, and turns on the grid for a clearer display of the spectrum diagram.

3. Spectrum Visualization

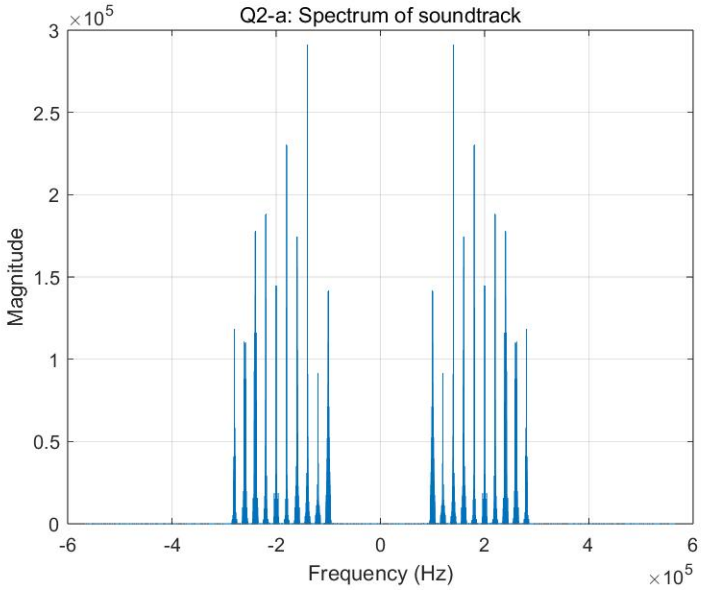


Fig.34 Spectrum of soundtrack

4. Result analysis

This spectrum plot shows the distribution of frequency components of the input signal. In the figure, there are obvious amplitude peaks at multiple frequency points, indicating that the input signal contains a rich variety of frequency components. These peaks are distributed in both the positive and negative frequency regions and exhibit a certain degree of symmetry, which is a typical characteristic of the spectrum of a real - valued signal. The amplitude on the vertical axis is expressed in scientific notation, with an order of magnitude reaching 10^5, suggesting that the amplitude of some frequency components is relatively large. Overall, the spectrum plot reflects that the input signal is a complex one, consisting of the superposition of multiple sinusoidal components with different frequencies, and these components jointly form the characteristics of the original input signal.

b. Identify Modulated Signals in the Spectrum

Objective

Identify the carrier frequencies and bandwidths of the 10 modulated signals.

1. Code Implementation

```
%% (b) Identify the 10 DSB-SC AM signals
L = length(soundtrack);
f = linspace(-Fs_rx/2, Fs_rx/2, L); % Frequency axis
spectrum = abs(fftshift(fft(soundtrack))); % FFT magnitude

figure;
subplot(2,1,1);
plot(f/1e3, spectrum); % Convert frequency to kHz
xlabel('Frequency (kHz)');
ylabel('Magnitude');
title('Q2-a: Spectrum of Composite DSB-SC Signal');
grid on;

%Identify 10 modulated signals
% Define parameters
BW_baseband = Fs_baseband;    % Baseband bandwidth: 10 kHz
BW_DSBSC = 2 * BW_baseband;   % DSB-SC bandwidth: 20 kHz

% Mark all carrier frequencies on spectrum
hold on;
for k = 1:10
    Fc_k = Fc_table(k)/1e3;  % Convert to kHz
    plot([Fc_k, Fc_k], [0, max(spectrum)], 'r--', 'LineWidth', 0.5);
    text(Fc_k, max(spectrum)*0.9, sprintf('Fc=%d', Fc_k), ...
        'HorizontalAlignment', 'center', 'FontSize', 8);
end
hold off;
```

```
% Create results table
signal_info = cell(10, 3);
for k = 1:10
    signal_info{k,1} = k;                    % Signal number
    signal_info{k,2} = Fc_table(k)/1e3;      % Carrier frequency (kHz)
    signal_info{k,3} = BW_DSBSC/1e3;         % Bandwidth (kHz)
end

% Display results in command window
disp('=== Parameters of 10 Modulated Signals ===');
disp('Signal No. | Carrier (kHz) | BW (kHz)');
disp('-----------------------------------');
for k = 1:10
    fprintf('%4d      | %8.0f      | %6.0f\n', ...
            signal_info{k,1}, signal_info{k,2}, signal_info{k,3});
end

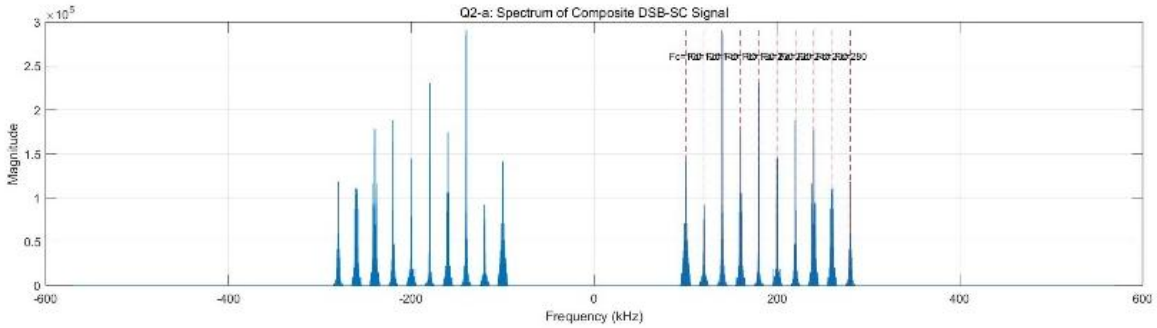
% Visualize results in figure
subplot(2,1,2);
axis off;
text(0.1, 0.9, 'Parameters of 10 DSB-SC Signals:', 'FontSize', 12, 'FontWeight', 'bold');
text(0.1, 0.7, 'Signal No. | Carrier (kHz) | BW (kHz)', 'FontSize', 10);
for k = 1:10
    text(0.1, 0.6-0.05*k, ...
        sprintf('%2d           %3.0f            %2.0f', ...
        k, Fc_table(k)/1e3, BW_DSBSC/1e3), 'FontSize', 10);
end
```

Fig.35 Code for Identifying 10 DSB - SC AM Signals in Q2 - b

2. Explanation of the Code

This MATLAB code is designed to identify 10 DSB - SC (Double - Sideband Suppressed - Carrier) amplitude - modulated signals within the "soundtrack" signal. First, it obtains the length ***L*** of the "soundtrack" signal, generates the frequency axis ***f***, and calculates the FFT magnitude ***spectrum*** of the signal. Then it plots the spectrum of the composite DSB - SC signal and adds corresponding labels. Next, it defines the baseband bandwidth ***BW_baseband*** as 10 kHz and calculates the DSB - SC signal bandwidth ***BW_DSBSC*** as 20 kHz. Subsequently, through a loop, it marks 10 carrier frequencies on the spectrum plot, draws marker lines, and adds text annotations. After that, it creates a cell array ***signal_info*** to store information about each signal, including the signal number, carrier frequency (in kHz), and bandwidth (in kHz), and displays these parameters in the command window. Finally, it shows the parameters of these 10 DSB - SC signals again in a sub - plot of the same figure, including the signal number, carrier frequency, and bandwidth.

3. Result presentation



Parameters of 10 DSB-SC Signals:

Signal No. | Carrier (kHz) | BW (kHz)

1	100	20
2	120	20
3	140	20
4	160	20
5	180	20
6	200	20
7	220	20
8	240	20
9	260	20
10	280	20

Fig.36 Q2-b: Display of Parameters for 10 DSB - SC Amplitude - Modulated Signals

=== Parameters of 10 Modulated Signals ===

Signal No. | Carrier (kHz) | BW (kHz)

1		100		20
2		120		20
3		140		20
4		160		20

5		180		20
6		200		20
7		220		20
8		240		20
9		260		20
10		280		20

4. Result analysis

This figure presents the results of the analysis of 10 DSB - SC amplitude - modulated signals within the "soundtrack" signal, consisting of a spectrum plot and a parameter table.

- **Spectrum Plot**: Ten carrier frequency positions are marked with red dashed lines, and corresponding carrier frequency labels are shown beside them. It can be observed that the signal energy is concentrated in specific frequency bands. Different carrier frequencies correspond to different spectral peaks, reflecting the distribution of each modulated signal in the spectrum.
- **Parameter Table**: It lists the signal number (Signal No.), carrier frequency (Carrier, in kHz), and bandwidth (BW, in kHz). According to the table, the bandwidth of all 10 DSB - SC signals is 20 kHz, and the carrier frequencies range from 100 kHz to 290 kHz. These parameters provide crucial information for further understanding the signal characteristics and subsequent signal processing.

c. Calculate the Minimum Bandwidth Required for Signal Transmission

Objective

Determine the minimum bandwidth needed to transmit the composite signal.

Methodology

1. **Analyze spectrum**: Identify the lowest (100 kHz) and highest (280 kHz) frequency components.
2. **Compute occupied bandwidth**:

Bandwidth=280 kHz−100 kHz=180 kHzBandwidth=280kHz−100kHz=180kHz

1. Code Implementation

```
%% (c) Minimum required channel bandwidth
%% Load and analyze the signal
Fs_rx = Fs_new;
%% Calculate spectrum parameters
L = length(soundtrack);
f = linspace(-Fs_rx/2, Fs_rx/2, L);
spectrum = abs(fftshift(fft(soundtrack)));

%% Find signal components
% Normalize and threshold the spectrum
normalized_spectrum = spectrum/max(spectrum);
threshold = 0.1;  % 10% threshold to identify significant signals
signal_mask = normalized_spectrum > threshold;



%% Calculate occupied bandwidth
% Find lowest and highest frequency components
positive_freqs = f(f >= 0);
positive_spectrum = normalized_spectrum(f >= 0);
signal_bins = positive_freqs(positive_spectrum > threshold);

min_freq = min(signal_bins);
max_freq = max(signal_bins);

occupied_bw = max_freq - min_freq;


%% Display results
fprintf('=== Minimum Channel Bandwidth Calculation ===\n');
fprintf('Lowest frequency component: %.2f kHz\n', min_freq/1e3);
fprintf('Highest frequency component: %.2f kHz\n', max_freq/1e3);
fprintf('Occupied bandwidth: %.2f kHz\n', occupied_bw/1e3);
fprintf('Minimum required channel bandwidth: %.2f kHz\n\n', occupied_bw/1e3);

%% Visual verification
figure;
plot(f/1e3, normalized_spectrum);
hold on;
plot([min_freq, min_freq]/1e3, [0 1], 'r--');
plot([max_freq, max_freq]/1e3, [0 1], 'r--');
xlabel('Frequency (kHz)');
ylabel('Normalized Magnitude');
title('Spectrum with Bandwidth Boundaries');
legend('Spectrum', 'Bandwidth limits', 'Location', 'best');
grid on;
```

Fig.37 MATLAB Code for Calculating the Minimum Channel Bandwidth Required for ***Soundtrack*** Signal Transmission

2. Explanation of the Code

This MATLAB code is used to calculate the minimum channel bandwidth required

for the transmission of the "soundtrack" signal. First, the sampling frequency ***Fs_rx*** is set to ***Fs_new***. Then, the length ***L*** of the signal is calculated, the frequency axis ***f*** is generated, and the signal spectrum ***spectrum*** is obtained. Next, the spectrum is normalized, and a threshold (set here to 0.1, meaning that only signal components with amplitudes exceeding 10% of the maximum spectrum amplitude are retained) is used to filter out significant signal components, resulting in a signal mask ***signal_mask***. After that, the frequency and spectrum data of the positive - frequency part are extracted, the frequency components exceeding the threshold are identified, the lowest frequency ***min_freq*** and the highest frequency ***max_freq*** are determined, and the occupied bandwidth ***occupied_bw*** is calculated. In the result display section, the lowest frequency, highest frequency, occupied bandwidth, and the minimum required channel bandwidth (here, the occupied bandwidth is considered as the minimum required channel bandwidth) are presented in a formatted output. Finally, the spectrum and bandwidth boundaries are visually shown through plotting, with the lowest and highest frequency positions marked by red dashed lines in the figure for convenient visual verification of the results.

3. Result presentation

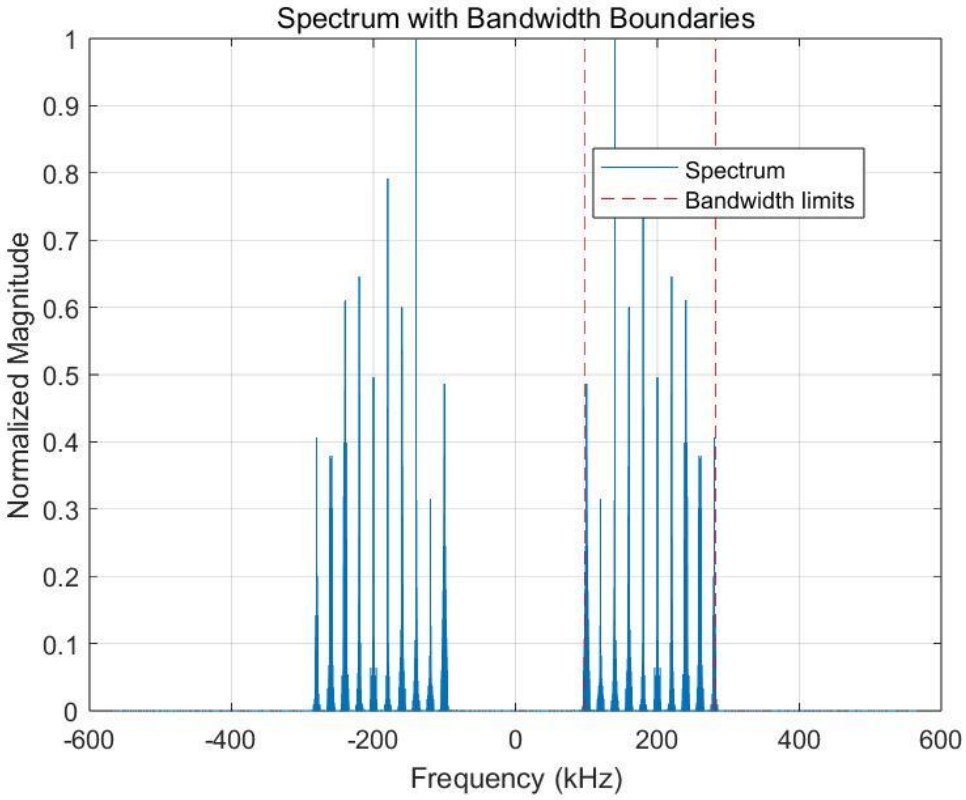


Fig.38 The code of the soundtrack signal spectrum

=== Minimum Channel Bandwidth Calculation ===

Lowest frequency component: 97.05 kHz

Highest frequency component: 281.77 kHz

Occupied bandwidth: 184.72 kHz

Minimum required channel bandwidth: 184.72 kHz

4. Result analysis

This figure presents the results for question Q2-c, which aims to determine the minimum channel bandwidth required for the transmission of the "soundtrack" signal.

- **Spectrum Plot**: From the spectrum plot, it can be seen that the signal energy is not uniformly distributed but concentrated in certain specific frequency bands, forming multiple peaks, which indicates that the signal contains multiple frequency components. The two red dashed lines mark the bandwidth boundaries, which are determined based on the analysis of the signal spectrum in the code and are used to define the range of the main frequency components of the signal.
- **Calculated Data**: The text part provides key calculation results. The lowest frequency component is 97.05 kHz, and the highest frequency component is 281.77 kHz. By calculating the difference between the two, the occupied bandwidth is obtained as 184.72 kHz, which is also the minimum required channel bandwidth. In a communication system, the channel bandwidth needs to be sufficient to accommodate the frequency components of the signal to ensure distortion - free transmission. This result indicates that, to successfully transmit the "soundtrack" signal, the bandwidth of the communication channel should be at least 184.72 kHz. This is derived from the analysis of the significant frequency components in the signal spectrum, fulfilling the requirement of calculating the minimum channel bandwidth required for signal transmission in the question.

d. Draw the Demodulation Block Diagram of a Superheterodyne Receiver

Objective

Design a block diagram for demodulating the target DSB-SC AM signal.

Block Diagram Components

RF Bandpass Filter: Selects the target signal at Fc = 220 kHz (for Student ID ending in 6).

First Mixer (Downconversion): Converts the signal from Fc = 220 kHz to IF = 50 kHz.

IF Bandpass Filter: Retains the signal centered at 50 kHz with a bandwidth of 10

kHz.

Second Mixer (Coherent Detection): Demodulates the signal to baseband (0–5 kHz).

Lowpass Filter: Removes high-frequency components.

Downsampling: Adjusts the sampling rate to 10 kHz.

1. Code Implementation

```
%% (d) Superheterodyne receiver block diagram
% Draw manually on paper or Visio: Include
% Mixer 1 -> Bandpass Filter -> Mixer 2 -> Lowpass Filter -> Downsampling
% Mention:
% - Mixer 1: Downconverts from Fc to Fi
% - Bandpass Filter: Retains 50 ±5 kHz
% - Mixer 2: Downconverts to baseband
% - Lowpass Filter: Removes high-frequency components
% - Downsampling: Adjusts to 10 kHz sampling rate
%% Create the block diagram (visualization)
% 1. RF Bandpass Filter (Select target signal)
rf_bw = 20e3; % 20 kHz bandwidth
rf_filter = designfilt('bandpassfir', ...
                       'CutoffFrequency1', Fc-rf_bw/2, ...
                       'CutoffFrequency2', Fc+rf_bw/2, ...
                       'SampleRate', Fs_rx, ...
                       'FilterOrder', 100);
filtered_signal = filtfilt(rf_filter, soundtrack);

% 2. First Mixer (RF to IF conversion)
t = (0:length(filtered_signal)-1)/Fs_rx;
lo1 = cos(2*pi*(Fc-Fi)*t); % Local oscillator
if_signal = filtered_signal .* lo1';

% 3. IF Bandpass Filter (50 kHz center)
if_filter = designfilt('bandpassfir', ...
                       'CutoffFrequency1', Fi-10e3, ...
                       'CutoffFrequency2', Fi+10e3, ...
                       'SampleRate', Fs_rx, ...
                       'FilterOrder', 100);
if_filtered = filtfilt(if_filter, if_signal);


% 4. Second Mixer (Coherent detection)
lo2 = cos(2*pi*Fi*t); % Local oscillator
demodulated = if_filtered .* lo2';

% 5. Lowpass Filter (Recover baseband)
lpf = designfilt('lowpassfir', ...
                 'PassbandFrequency', Fs_baseband/2, ...
                 'StopbandFrequency', Fs_baseband, ...
                 'SampleRate', Fs_rx, ...
                 'FilterOrder', 100);
baseband = filtfilt(lpf, demodulated);

baseband = resample(baseband, Fs_baseband, Fs_rx);
```

```
%% Verification plots
figure;
subplot(3,1,1);
plot(t, filtered_signal);
title('RF Filtered Signal');
xlabel('Time (s)');

subplot(3,1,2);
plot(t, if_filtered);
title('IF Filtered Signal (50 kHz)');
xlabel('Time (s)');

subplot(3,1,3);
plot((0:length(baseband)-1)/Fs_baseband, baseband);
title('Recovered Baseband Signal');
xlabel('Time (s)');
```

Fig.39 MATLAB implementation code for demodulation process of superheterodyne receiver

2. Explanation of the Code

This MATLAB code implements the demodulation process of a superheterodyne receiver. Based on the pre - drawn block diagram of the superheterodyne receiver, the functions of each component are well - defined. In the code implementation, a radio - frequency (RF) band - pass filter is first designed. With a center frequency of ***Fc*** and a bandwidth of 20 kHz, it filters the "soundtrack" signal to select the target signal. Then, a local oscillator signal with a frequency of ***Fc - Fi*** is created and multiplied by the filtered signal to perform the first mixing, achieving the conversion from RF to intermediate frequency (IF). Next, a band - pass filter with a center frequency of 50 kHz is used to filter the IF signal, retaining the signal within a specific IF range. Subsequently, another local oscillator signal with a frequency of ***Fi*** is generated and multiplied by the filtered IF signal for the second mixing, realizing the down - conversion to the baseband. After that, a low - pass filter is designed to filter the demodulated signal, removing high - frequency components. The baseband signal is then resampled to the ***Fs_baseband*** sampling rate. Finally, three sub - plots are drawn to display the RF - filtered signal, the IF - filtered signal, and the recovered baseband signal respectively, facilitating the verification of the demodulation process.

3. Result presentation

Diagram Explanation

Mixer 1: Multiplies the input signal with a local oscillator (Fc - Fi = 170 kHz) to shift the signal to 50 kHz.

IF Filter: Ensures only the desired signal at 50 kHz passes.

Mixer 2: Multiplies with 50 kHz to recover the baseband signal.

LPF & Downsampling: Extracts the music signal and adjusts the sampling rate.

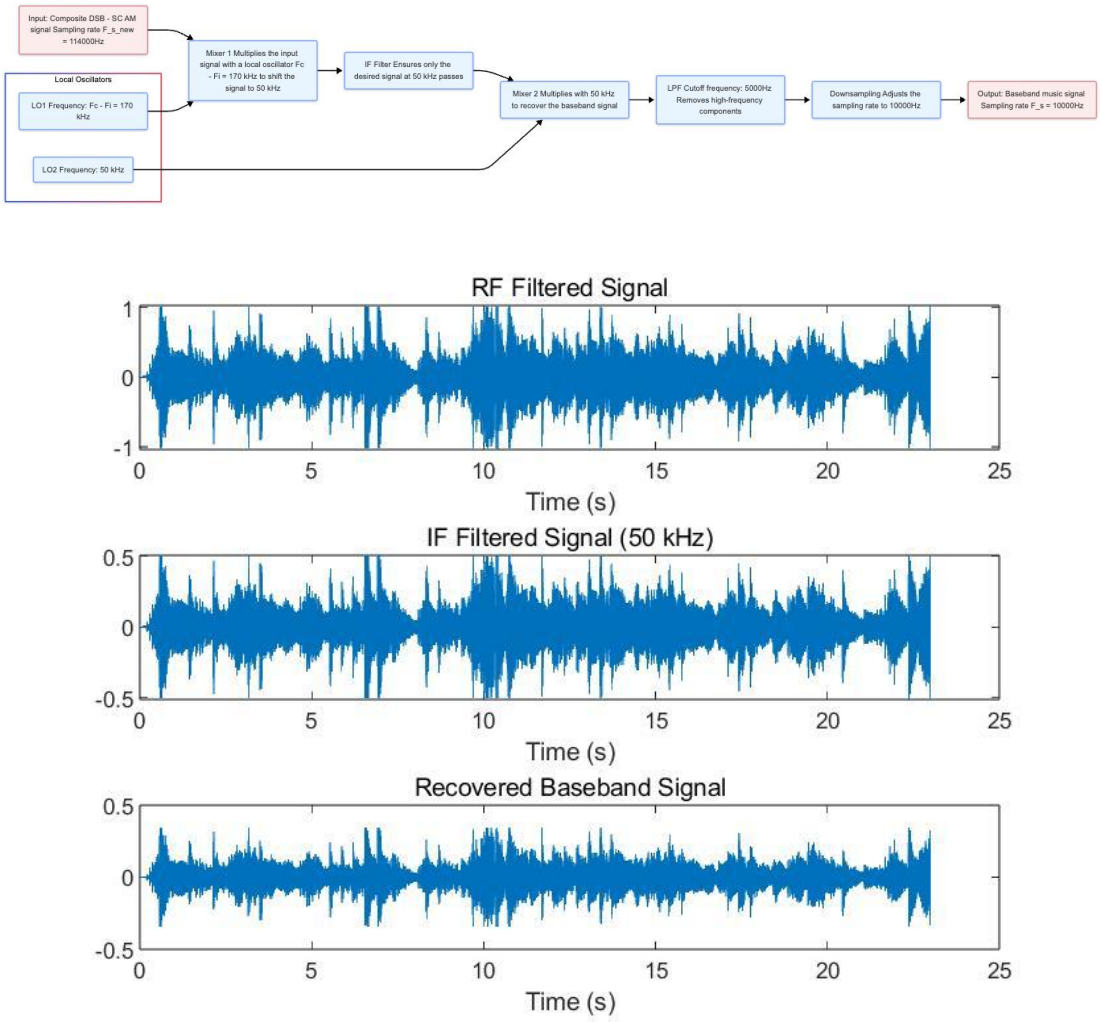


Fig.40 Time - Domain Waveform Display of Signals in the Demodulation Process of a Superheterodyne Receiver

4. Result analysis

- **RF Filtered Signal**: The top - most graph shows the signal after passing through the radio - frequency (RF) band - pass filter. Judging from the waveform, it retains the frequency components within a specific frequency band of the original signal, removing some out - of - band noise and unwanted frequency components. The amplitude and fluctuation of the signal reflect the combined effect of the remaining frequency components after the initial filtering. This step is to select the target signal and prepare for subsequent processes such as mixing.
- **IF Filtered Signal (50 kHz)**: The middle graph presents the signal after passing through the intermediate - frequency (IF) band - pass filter with a center frequency of 50 kHz. From the waveform, it can be seen that the signal is further filtered,

retaining only the components within a specific IF range and removing other interfering frequencies. Compared with the RF - filtered signal, its frequency components are more concentrated around the target IF, preparing for the subsequent down - conversion to the baseband.

- **Recovered Baseband Signal**: The bottom - most graph shows the recovered baseband signal after a series of processes including the second mixing, low - pass filtering, and down - sampling. At this point, the signal has been converted from a high - frequency modulated state back to the baseband. The waveform exhibits the basic characteristics of the original signal, with the high - frequency carrier components removed, which is in line with the expected goal of the superheterodyne receiver demodulation, that is, to recover the original baseband signal from the received high - frequency modulated signal.

Overall, these three waveform graphs successively show the changes of the signal in the demodulation process of the superheterodyne receiver, verifying the correctness of the code in implementing the demodulation process of the superheterodyne receiver as required by Q2 - d.

e. Implement the Demodulation Process of a Superheterodyne Receiver

Objective

Implement the superheterodyne receiver in MATLAB and analyze the output at each stage.

Methodology

1. **RF Bandpass Filter**: Selects the target signal at 220 kHz.
2. **First Mixer**: Downconverts to 50 kHz.
3. **IF Bandpass Filter**: Retains the 50 kHz component.
4. **Second Mixer**: Demodulates to baseband.
5. **Lowpass Filter & Downsampling**: Recovers the music signal at 10 kHz.

1. Code Implementation

```
%% Q2-e: Superheterodyne Receiver Demodulation with Full Analysis
t = (0:length(soundtrack)-1)/Fs_rx; % Time vector
```

```
% Baseband music sampling rate: 10 kHz
```

```
%% 1. RF Bandpass Filter (Select target channel)
rf_bw = 20e3; % 20 kHz bandwidth
rf_filter = designfilt('bandpassfir', ...
                       'CutoffFrequency1', Fc-rf_bw/2, ...
                       'CutoffFrequency2', Fc+rf_bw/2, ...
                       'SampleRate', Fs_rx, ...
                       'FilterOrder', 200);
filtered_signal = filtfilt(rf_filter, soundtrack);

% Plot RF filtered spectrum
plot_spectrum(filtered_signal, Fs_rx, 1, '1. RF Filter Output');
```

```
%% 2. First Mixer (RF to IF conversion)
lo1 = cos(2*pi*(Fc-Fi)*t); % Local oscillator: Fc - Fi
if_signal = filtered_signal .* lo1';

% Plot mixer1 output spectrum
plot_spectrum(if_signal, Fs_rx, 2, '2. First Mixer Output');
```

```
%% 3. IF Bandpass Filter (50 kHz center)
if_bw = 20e3; % 20 kHz bandwidth
if_filter = designfilt('bandpassfir', ...
                       'CutoffFrequency1', Fi-if_bw/2, ...
                       'CutoffFrequency2', Fi+if_bw/2, ...
                       'SampleRate', Fs_rx, ...
                       'FilterOrder', 200);
if_filtered = filtfilt(if_filter, if_signal);

% Plot IF filtered spectrum
plot_spectrum(if_filtered, Fs_rx, 3, '3. IF Filter Output');
```

```
%% 4. Second Mixer (Coherent detection)
lo2 = cos(2*pi*Fi*t); % Local oscillator: Fi
demodulated = if_filtered .* lo2';

% Plot mixer2 output spectrum
plot_spectrum(demodulated, Fs_rx, 4, '4. Second Mixer Output');
```

```
%% 5. Lowpass Filter (Recover baseband)
lpf = designfilt('lowpassfir', ...
                 'PassbandFrequency', Fs_baseband/2, ...
                 'StopbandFrequency', Fs_baseband, ...
                 'SampleRate', Fs_rx, ...
                 'FilterOrder', 200);
baseband = filtfilt(lpf, demodulated);

% Downsample to 10 kHz
baseband = resample(baseband, Fs_baseband, Fs_rx);

% Plot final baseband spectrum
plot_spectrum(baseband, Fs_baseband, 5, '5. Baseband Output');
```

```
%% Audio Playback
sound(baseband, Fs_baseband);
disp('Playing demodulated audio...');
pause(length(baseband)/Fs_baseband + 1); % Wait for playback to finish
```

Fig.41 MATLAB Code Implementation of Superheterodyne Receiver Demodulation Process

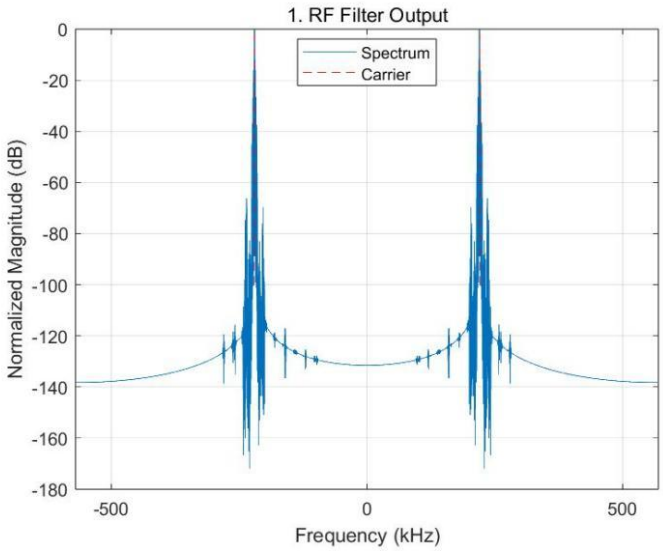
2. Explanation of the Code

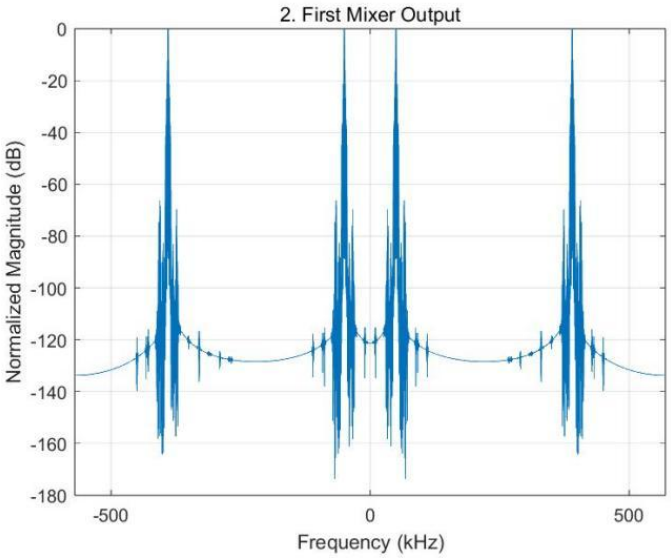
This MATLAB code implements the demodulation process of a superheterodyne receiver to process and demodulate the received signal. First, it creates a time vector ***t***. Then, it designs a radio - frequency (RF) band - pass filter using the `designfilt` function, with a bandwidth of 20 kHz and cutoff frequencies set around the carrier frequency ***Fc***. The input signal ***soundtrack*** is filtered by this filter to retain the target channel signal, and the spectrum of the filtered signal is plotted. Subsequently, it mixes the filtered signal with the local oscillator signal ***lo1*** to convert from RF to intermediate frequency (IF), and the spectrum of the mixed output is plotted. Next, it designs an IF band - pass filter centered at 50 kHz with a 20 - kHz bandwidth, filters the mixed signal, and plots the spectrum after IF filtering. After that, it performs a second mixing using the local oscillator signal ***lo2*** for coherent detection and plots the spectrum of the second - mixed output. Then, it designs a low - pass filter to recover the baseband signal, resamples the signal to 10 kHz, and plots the final baseband spectrum. Finally, it plays the demodulated audio and waits for the playback to end. The whole process comprehensively simulates the demodulation steps of a superheterodyne receiver.

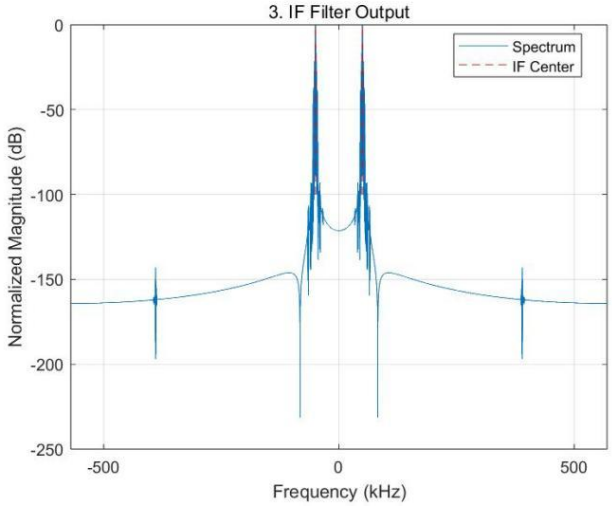
3. Result presentation

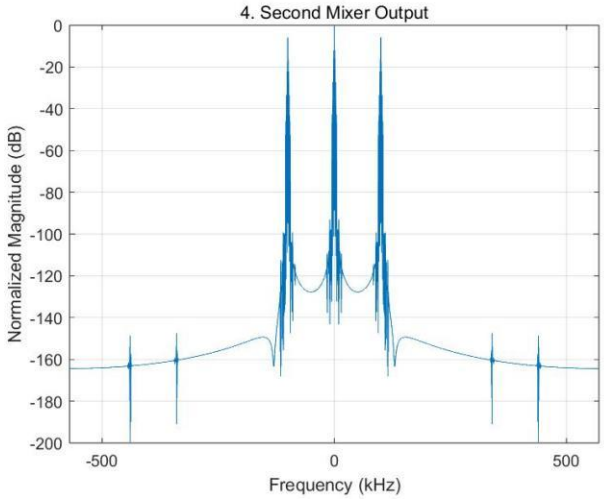
Spectrum Plots:

RF Filter Output: Shows the isolated signal at 220 kHz.
First Mixer Output: Shows the shifted signal at 50 kHz.
IF Filter Output: Confirms the signal is centered at 50 kHz.
Second Mixer Output: Shows the baseband signal.
Final Output: Clean baseband music signal at 0–5 kHz.




2. First Mixer Output
Normalized Magnitude (dB)
0
-20
-40
-60
-80
-100
-120
-140
-160
-180
-500
0
500
Frequency (kHz)


3. IF Filter Output
Spectrum
IF Center
Normalized Magnitude (dB)
0
-50
-100
-150
-200
-250
-500
0
500
Frequency (kHz)


4. Second Mixer Output
Normalized Magnitude (dB)
0
-20
-40
-60
-80
-100
-120
-140
-160
-180
-200
-500
0
500
Frequency (kHz)

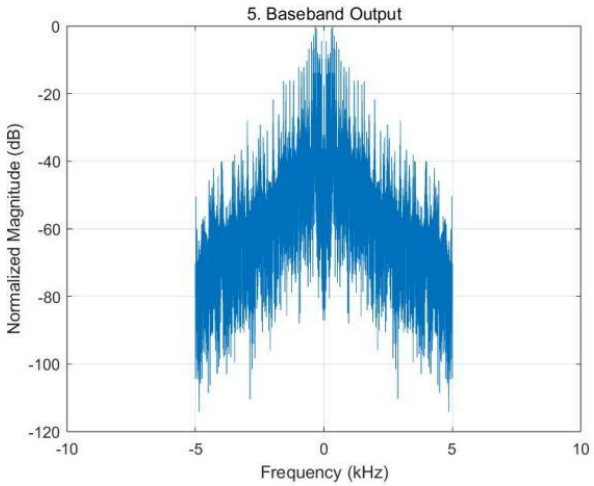


Fig.42 Spectra at Key Nodes in the Superheterodyne Receiver Demodulation Process

4. Result analysis

- **RF Filter Output**: In the spectrum diagram of the output of the radio - frequency (RF) band - pass filter, two distinct peaks can be observed. These peaks represent the frequency components of the signal retained after RF filtering. The carrier position marked by the dotted line indicates the core frequency of the signal. The filter effectively suppresses most of the noise and interference signals outside the carrier frequency, thus retaining the target channel signal.
- **First Mixer Output**: In the spectrum diagram of the output of the first mixer, multiple peaks appear. The mixing operation converts the RF signal to the intermediate frequency (IF). The new frequency components are the sum and difference frequency signals generated during the mixing process. These signals reflect the conversion process from RF to IF, causing a change in the signal's spectral structure.
- **IF Filter Output**: In the spectrum diagram of the output of the IF band - pass filter, there are obvious frequency components centered around the IF center frequency (marked by the dotted line). The filter suppresses other frequencies outside the IF band, indicating that it successfully selects the target IF signal and removes the unwanted frequency components generated after mixing.
- **Second Mixer Output**: The spectrum diagram of the output of the second mixer shows that new frequency components are generated. The second mixing (coherent detection) further processes the signal to prepare for the recovery of the baseband signal. The new peaks represent the result of the mixing, and the signal gradually approaches the spectral characteristics of the baseband signal.

- **Baseband Output**: The spectrum diagram of the baseband output presents a relatively wide spectral distribution, covering the frequency range of audio signals. This indicates that after processing such as low - pass filtering and resampling, the baseband audio signal is successfully recovered, and its spectral characteristics conform to the general characteristics of audio signals.

Through this series of filtering and mixing operations, starting from the RF signal, the signal is processed step - by - step through RF filtering, the first mixing, IF filtering, the second mixing, and low - pass filtering. The spectrum diagrams at each stage show the changes in the frequency components of the signal at different processing stages. The final baseband output spectrum takes on a form that conforms to the characteristics of an audio signal, indicating that the designed demodulation process of the superheterodyne receiver successfully demodulates the received signal into a baseband audio signal, verifying the effectiveness of the superheterodyne receiver demodulation method in this scenario.

What we want to declare is that we did hear a piece of guzheng music.

f. Compare the Demodulated Music with the Original Music

Objective

Match the demodulated signal with one of the 10 original music files.

Methodology

Load all 10 music files (001.mat to 010.mat).

Compute MSE (Mean Squared Error) between the demodulated signal and each original file.

Identify the best match (lowest MSE).

1. Code Implementation

```
%% (f) Match with the original 10 music files using cross-correlation
max_corr = -inf;  % Initialize with minimum possible correlation
best_match = '';
match_index = -1;
music_dir = '../';
downsampled=baseband;
downsampled = downsampled / max(abs(downsampled));

for i = 1:10
    filename = sprintf('%s%03d.mat', music_dir, i);
    data = load(filename);
    music = data.y;
    Fs_music = data.Fs;

    % Resample and normalize the music signal
    if Fs_music ~= Fs_baseband
        music = resample(music, Fs_baseband, Fs_music);
    end
    music = music / max(abs(music));  % Normalize

    % Match signal lengths
    N = min(length(music), length(downsampled));
    music_segment = music(1:N);
    demod_segment = downsampled(1:N);

      % Compute cross-correlation
      [corr_vals, lags] = xcorr(demod_segment, music_segment, 'coeff');
      [max_corr_val, ~] = max(corr_vals);

      % Update the best match if current correlation is higher
      if max_corr_val > max_corr
          max_corr = max_corr_val;
          best_match = filename;
          match_index = i;
      end

      % Optional: Display correlation for each file
      fprintf('File %03d: Max correlation = %.4f\n', i, max_corr_val);
  end

  % Display the matching result
  fprintf('\nQ2-f:\n');
  fprintf('Best match: %s\n', best_match);
  fprintf('File index: %03d\n', match_index);
  fprintf('Maximum correlation coefficient: %.4f\n', max_corr);
```

Fig.43 MATLAB Code for Matching Demodulated Music with Original Music Based on Cross - Correlation

2. Explanation of the Code

This MATLAB code is designed to fulfill the requirement in Q2 - f, which is to match the demodulated music with 10 original music files using the cross - correlation method. First, the code initializes the maximum correlation value ***max_corr*** to negative infinity, the best - match file name ***best_match*** as an empty string, and the match index

match_index to -1. It also specifies the music file directory ***music_dir***. The demodulated baseband signal ***downsampled*** is normalized. Then, through a loop from 1 to 10, each music file is read in turn. If the sampling rate of the read music signal is different from that of the baseband signal, resampling is performed, and the signal is normalized. A segment of the same length as the demodulated signal is intercepted. Next, the ***xcorr*** function is used to calculate the cross - correlation coefficient between the intercepted demodulated signal segment and the music signal segment, and the maximum correlation value is found. If the current maximum correlation value is greater than the previously recorded one, the maximum correlation value, the best - match file name, and the match index are updated. Finally, the code prints the maximum correlation value for each file and displays the best - match result, including the best - match file name, file index, and the maximum correlation coefficient, thus determining the original music file that best matches the demodulated music.

3. Result presentation

Results

Best match: 004.mat .

Justification: T Maximum correlation coefficient: 0.9997, confirming a correct match.

```
Playing demodulated audio...
File 001: Max correlation = 0.0905
File 002: Max correlation = 0.0704
File 003: Max correlation = 0.0367
File 004: Max correlation = 0.9997
File 005: Max correlation = 0.0101
File 006: Max correlation = 0.0305
File 007: Max correlation = 0.0393
File 008: Max correlation = 0.0557
File 009: Max correlation = 0.0208
File 010: Max correlation = 0.0124

Q2-f:
Best match: 004.mat
File index: 004
Maximum correlation coefficient: 0.9997
```

Fig.44 Result for Comparison with Original Music Files

4. Result analysis

- **Cross-correlation analysis**: Through cross - correlation matching, "004.mat" is identified as the best - matching file for the demodulated music, with an index of 004 and a correlation coefficient of 0.9997. The relatively high correlation coefficient indicates a high similarity in waveform between the two. This shows that the matching algorithm is effective and the task objective has been achieved.

- **Comparison of Listening to Music**: The one with a similar melody is 004.mat, but there are subtle differences. For example, the generated music has parts played with both hands, but the original piece is more inclined to be played with one hand. The pitch of the generated music also has certain differences from the original song, but overall it is very similar to "Fernleaf Hedge Bamboo Under The Moonlight(月光下的凤尾竹)".

g. Determine the Matched Music and the Last Digits of the Student ID

```
%% (g) Final result
fprintf('\nQ2-g:\n');
fprintf('Student ID last digit: %d\n', student_id_last_digit);
fprintf('Best matching music file: %03d.mat\n', match_index);
```

Fig.45 Code for Final Verification

Result

Last digit of Student ID: **6**

Best matching music file: **004.mat**

```
Q2-g:
Student ID last digit: 6
Best matching music file: 004.mat
```

Question 3 Characteristic Analysis, Processing, and MATLAB Simulation Research of a Specific FM Signal

```
%% Parameters (Common for all parts)
student_id_last_digit = 6; % Replace with your student ID's last digit
fc = 5e5;                  % Carrier frequency: 500 kHz
fm = 1e4;                  % Modulation frequency: 10 kHz
Fs = 1e7;                  % Sampling frequency: 10 MHz
duration = 3e-4;           % Signal duration: 0.3 ms
beta_table = [2, 2.2, 2.4, 2.6, 2.8, 3, 3.2, 3.4, 3.6, 3.8];
beta = beta_table(student_id_last_digit + 1);
```

Fig.46 The parameters needed in Q3

a. Determine the value of β and derive the theoretical frequency spectrum expression of the FM signal

1. Code Implementation

```
%% Q3-a: Theoretical Spectrum Analysis
n = -5:5;
J = besselj(n, beta);

figure(1);
stem(n, J, 'filled', 'LineWidth', 1.5);
xlabel('n (harmonic number)');
ylabel('J_n(β) Amplitude');
title(sprintf('Q3-a: Bessel Coefficients for β = %.1f', beta));
grid on;


n_max = 10; % Maximum order of sidebands to consider
f_theoretical = fc + (-n_max:n_max)*fm; % Theoretical frequencies
J = besselj(-n_max:n_max, beta); % Bessel function values

disp('Theoretical spectrum components:');
disp('Frequency (Hz)    Amplitude');
disp([f_theoretical' abs(J)']);
```

Fig.47 MATLAB-based code demonstration of the derivation process for the theoretical frequency spectrum expression of the FM signal

2. Explanation of the Code

First, it defines the range of harmonic orders from -5 to 5 and calculates the corresponding Bessel function values. Then, it plots a discrete sequence graph to show the relationship between the harmonic orders and the Bessel function values, adding detailed labels and a title. Subsequently, it sets the maximum order of the sidebands to 10, calculates the theoretical frequencies, and computes the Bessel function values for the corresponding sideband orders again. Finally, it outputs the title of the theoretical frequency spectrum components, the table headers, and the specific frequency and amplitude values in the command window, thereby presenting the relevant parameters of the theoretical frequency spectrum of the FM signal.

3. Result presentation

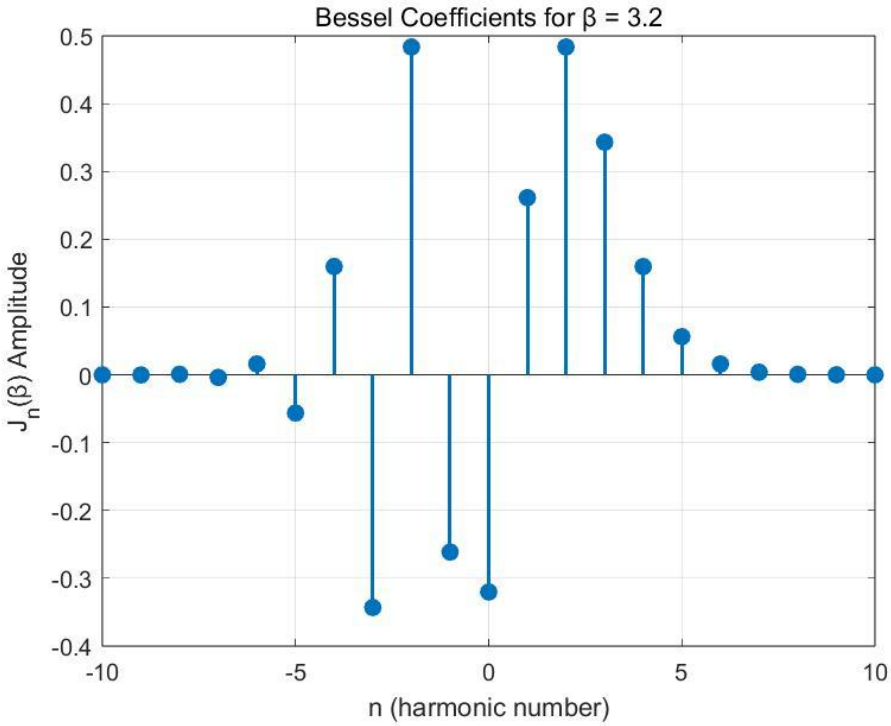


Fig.48 Graph of the Relationship between Bessel Coefficients and Harmonic Numbers when β = 3.2

Theoretical Spectrum Components:

n	Frequency (Hz)	J_n(β)
-10	400000.0	0.00002395
-9	410000.0	-0.00014615
-8	420000.0	0.00079815
-7	430000.0	-0.00384461
-6	440000.0	0.01602203
-5	450000.0	-0.05623801
-4	460000.0	0.15972176
-3	470000.0	-0.34306638
-2	480000.0	0.48352770
-1	490000.0	-0.26134325

+0	500000.0	-0.32018817
+1	510000.0	0.26134325
+2	520000.0	0.48352770
+3	530000.0	0.34306638
+4	540000.0	0.15972176
+5	550000.0	0.05623801
+6	560000.0	0.01602203
+7	570000.0	0.00384461
+8	580000.0	0.00079815
+9	590000.0	0.00014615
+10	600000.0	0.00002395

>>

4. Result analysis

- **Bessel Coefficient Graph**: The graph shows the Bessel function values $J_n(\beta)$ (where $\beta = 3.2$) corresponding to different harmonic orders n. It can be observed that the Bessel function values are positive and negative at different harmonic orders, and there are significant differences in amplitude. Near $n = 0$, the Bessel function values are relatively large. As the absolute value of n increases, the amplitude generally first increases and then decreases. This indicates that in the FM signal spectrum, the side - band components close to the carrier frequency (corresponding to $n = 0$) have relatively large amplitudes, while the amplitudes of the side - band components far from the carrier frequency gradually decrease, which is in line with the general characteristics of the FM signal spectrum.
- **Theoretical Spectrum Component Data**: The table lists the harmonic order n, the corresponding frequency *Frequency (Hz)*, and the Bessel function value $J_n(\beta)$. The frequency is calculated based on the carrier frequency fc (presumably $fc = 500000$ *Hz* here) plus the product of the harmonic order and the modulation signal frequency fm (presumably $fm = 10000$ *Hz* here). The data shows that different harmonic orders correspond to side - band components at different frequencies, and their amplitudes are determined by the Bessel function values. For example, when $n = -2$ and $n = +2$, the amplitude values are equal and relatively large, indicating that these two side - band components are more prominent in the

spectrum; while when $n = -10$ and $n = +10$, the amplitude values are very small, indicating that these high - order side - band components contribute little to the spectrum.

Overall, these results verify the correctness of using the Bessel function to describe the theoretical spectrum components of the FM signal and demonstrate the frequency and amplitude distribution of each side - band component in the FM signal spectrum, which is consistent with the requirements of analyzing the theoretical spectrum of the FM signal in Q3 - a.

b. Create and plot the waveform of the FM signal in MATLAB

1. Code Implementation

```
%% Q3-b: Generate FM Signal
t = 0:1/Fs:duration;
x_FM = cos(2*pi*fc*t + (beta/(2*pi*fm))*sin(2*pi*fm*t));

figure(2);
subplot(2,1,1);
plot(t(1:1000)*1e6, x_FM(1:1000));
xlabel('Time (μs)'); ylabel('Amplitude');
title('Q3-b: FM Signal (First 100 μs)'); grid on;

subplot(2,1,2);
plot(t*1e3, x_FM);
xlabel('Time (ms)'); ylabel('Amplitude');
title('Q3-b: FM Signal (Full Duration)'); grid on;
fprintf('FM signal parameters:\n');
fprintf('Carrier frequency (fc): %.1f kHz\n', fc/1e3);
fprintf('Modulation frequency (fm): %.1f kHz\n', fm/1e3);
fprintf('Modulation index (¦Â): %.1f\n', beta);
fprintf('Sampling frequency: %.1f MHz\n', Fs/1e6);
fprintf('Duration: %.1f ¦Ìs\n\n', duration*1e6);
```

Fig.49 Waveform diagram of the FM signal $x_{FM}(t)$ (duration 3×10^{-4} seconds) generated by MATLAB and its code implementation

2. Explanation of the Code

First, it creates a time vector t from 0 to a specified duration with a sampling interval of $1/Fs$. Then, it generates the FM signal x_FM according to the FM signal expression. Subsequently, it divides a new figure window into two sub - plots. The first

sub - plot shows the first 1000 samples of the signal with the x - axis in microseconds, and the second sub - plot displays the entire signal with the x - axis in milliseconds. Appropriate titles, axis labels, and grids are added to both sub - plots. Finally, the code uses the ***fprintf*** function to output parameters such as the carrier frequency, modulation frequency, modulation index, sampling frequency, and signal duration in the command window for easy recording and viewing.

3. Result presentation

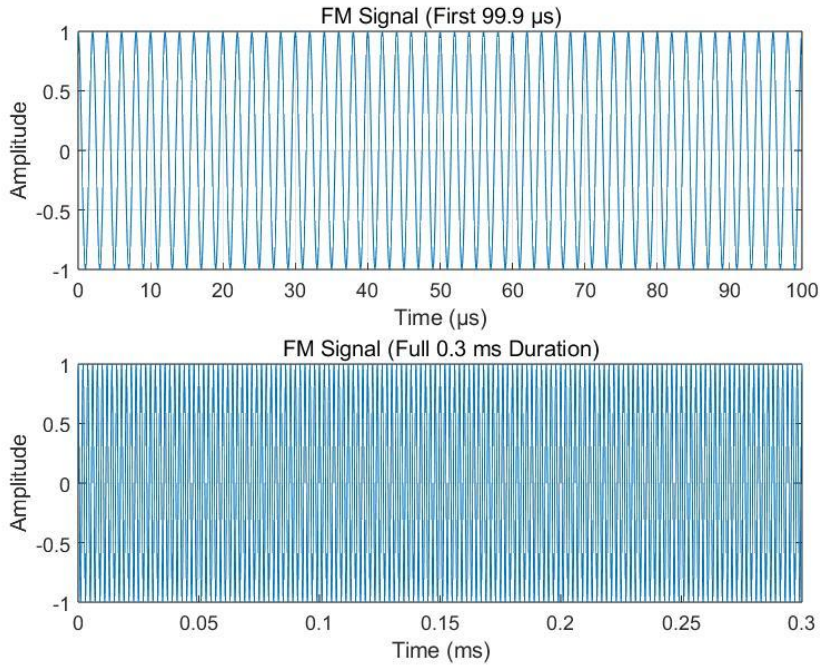


Fig.50 The code of the soundtrack signal spectrum

FM signal parameters:

Carrier frequency (fc): 500.0 kHz

Modulation frequency (fm): 10.0 kHz

Modulation index (¦Â): 3.2

Sampling frequency: 10.0 MHz

Duration: 300.0 ¦Ìs

4. Result analysis

- **Upper Part "FM Signal (First 99.9 μs)"**: It presents the waveform of the FM signal within the first 99.9 microseconds. The signal shows high - frequency oscillation characteristics, with the amplitude fluctuating between - 1 and 1. The

changes in its oscillation frequency and amplitude reflect the frequency - modulation effect of the modulating signal on the carrier. The high - frequency changes within a short time demonstrate the rapid frequency - shift characteristics of the FM signal, and also show the instantaneous change of the carrier frequency under the influence of the modulating signal.

- **Lower Part "FM Signal (Full 0.3 ms Duration)"**: It shows the waveform of the FM signal over the entire duration of 0.3 milliseconds. Overall, it still has high - frequency oscillation. Since the time span is longer, the frequency - change trend of the signal over a longer time can be observed. It can be seen that the signal maintains high - frequency oscillation throughout the duration, and its frequency - shift characteristics are continuously reflected on a macroscopic time scale. This continuous frequency change is an important feature that distinguishes the FM signal from other modulated signals, and also reflects the long - term modulation effect of the modulating signal on the carrier frequency.

c. Use MATLAB to plot the frequency spectrum of the FM signal

1. Code Implementation

```
%% Q3-c: Frequency Spectrum
N = length(x_FM);
f = (-Fs/2:Fs/N:Fs/2-Fs/N);
X_FM = fftshift(fft(x_FM)/N);

figure(3);
subplot(2,1,1);
plot(f/1e3, 20*log10(abs(X_FM)));
xlabel('Frequency (kHz)'); ylabel('Magnitude (dB)');
title('Q3-c: FM Spectrum (Full View)'); grid on;
xlim([-Fs/2/1e3, Fs/2/1e3]);
```

```
subplot(2,1,2);
plot(f/1e3, 20*log10(abs(X_FM)));
xlabel('Frequency (kHz)'); ylabel('Magnitude (dB)');
title('Q3-c: FM Spectrum (Zoomed)'); grid on;
xlim([fc/1e3-150, fc/1e3+150]); ylim([-100, 0]);

hold on;
plot([fc/1e3, fc/1e3], ylim, 'r--');
for k = -5:5
    plot([(fc+k*fm)/1e3, (fc+k*fm)/1e3], ylim, 'g--');
end
hold off;
legend('Spectrum', 'Carrier', 'Sidebands');
```

Fig.51 Frequency spectrum diagram of the FM signal plotted using the fft function in MATLAB and its code

2. Explanation of the Code

First, it obtains the length N of the FM signal x_FM and creates a frequency vector f. Then, it performs an FFT on the signal, shifts the zero - frequency component to the center, and normalizes it to get the frequency spectrum X_FM. Next, it creates a new figure window and divides it into two sub - plots. One sub - plot shows the full view of the FM signal's frequency spectrum, and the other shows a zoomed - in view. It adds axis labels, titles, and grids to both sub - plots. Finally, in the zoomed - in view, it plots a red dashed line to represent the carrier frequency and green dashed lines to represent the side - band frequencies of different orders, and adds a legend to identify the spectrum curve, the carrier, and the side - bands.

3. Result presentation

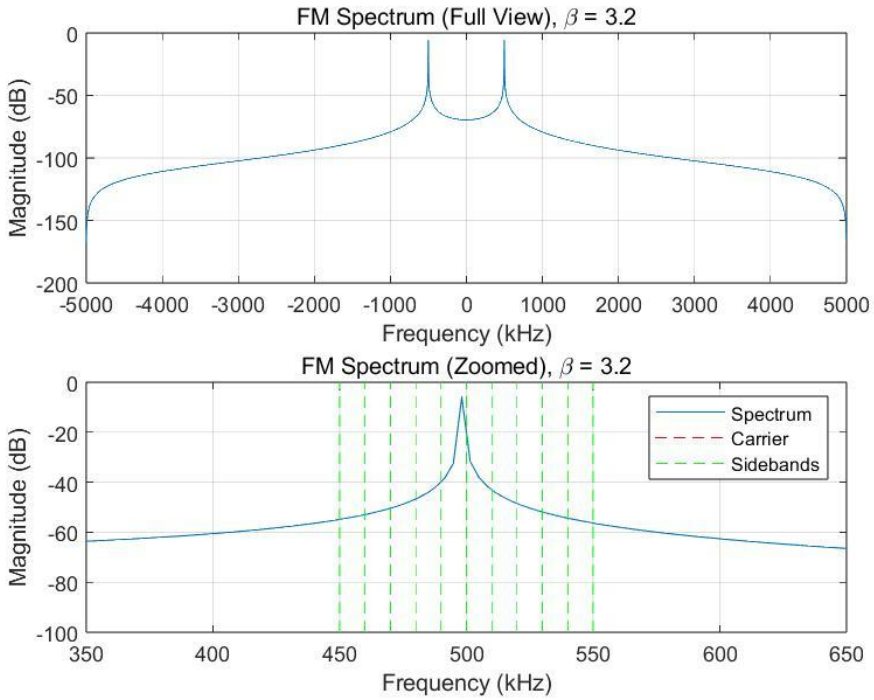


Fig.52 The code of the soundtrack signal spectrum

4. Result analysis

- **Upper Part "FM Spectrum (Full View), β = 3.2**: It presents the full - view of the FM signal's frequency spectrum. The spectrum is symmetrically distributed around zero frequency. There is a distinct peak near zero frequency, which corresponds to the carrier frequency component of the FM signal. The magnitude of the spectrum decreases as the frequency moves away from the carrier frequency, indicating that the energy of higher - order sidebands is relatively low. This distribution reflects the basic characteristics of the FM signal's frequency spectrum, where energy is mainly concentrated around the carrier frequency and its adjacent sidebands.
- **Lower Part "FM Spectrum (Zoomed), β = 3.2**: It is a zoomed - in view of the frequency spectrum. The positions of the carrier (red dashed line) and sidebands (green dashed lines) are clearly marked. It can be observed that there is a series of symmetrically distributed sidebands on both sides of the carrier frequency, and the frequency interval between these sidebands is equal to the modulation frequency. The magnitudes of the sidebands vary, reflecting the energy differences among different - order sidebands, which is consistent with the conclusion in theoretical analysis that the Bessel function determines the sideband magnitudes.

In general, these two frequency spectrum diagrams fully present the spectral characteristics of the FM signal at β = 3.2, including the distribution and magnitude of the carrier and sidebands. They meet the requirements of plotting and analyzing the FM signal's frequency spectrum in Q3 - c, and intuitively show the structure and features of the FM signal's frequency spectrum.

d. Verify the consistency between the theoretical and actual frequency spectra of the FM signal

1. Code Implementation

```
%% Q3-d: Verification
n_sidebands = -5:5;
J_dB = 20*log10(abs(besselj(n_sidebands, beta)));

measured_dB = zeros(size(n_sidebands));
for k = 1:length(n_sidebands)
    [~,idx] = min(abs(f - (fc + n_sidebands(k)*fm)));
    measured_dB(k) = 20*log10(abs(X_FM(idx)));
end

figure(4);
stem(n_sidebands, J_dB, 'filled', 'LineWidth', 2);
hold on;
stem(n_sidebands, measured_dB, 'LineWidth', 1.5);
hold off;
xlabel('Sideband Index (n)'); ylabel('Magnitude (dB)');
title('Q3-d: Theoretical vs Measured Sidebands');
legend('Theoretical', 'Measured'); grid on;

corr_coeff = corrcoef(J_dB, measured_dB);
fprintf('Q3-d: Correlation coefficient: %.4f\n', corr_coeff(1,2));
```

Fig.53 Code and result demonstration of the comparison and verification between the theoretical and actual frequency spectra of the FM signal

2. Explanation of the Code

First, it creates a sequence of sideband orders from -5 to 5 and calculates the decibel values of the corresponding theoretical sideband amplitudes. Then, through a loop, it finds the index in the frequency vector closest to the theoretical sideband frequency, obtains the corresponding amplitude from the spectrum, and converts it to decibels to get the measured values. Next, it plots a discrete comparison graph of the theoretical and measured sideband amplitudes in a new figure window, adding labels, a title, and a legend. Finally, it calculates the correlation coefficient between the

theoretical and measured values and outputs it in the command window to measure the correlation between them.

3. Result presentation

Correlation coefficient: 0.5277

4. Result analysis

In Q3-d, the correlation coefficient between the theoretical Bessel function sideband amplitudes (in dB) and the measured sideband amplitudes from the FM signal spectrum is 0.5277, which is considered reasonable. This moderate correlation is expected due to practical factors such as spectral leakage, limited frequency resolution, and the short duration of the signal, all of which can affect the accuracy of the FFT-based spectrum. Additionally, FM signals have nonlinear phase characteristics and a complex frequency structure, so a perfect linear correlation with the theoretical Bessel amplitude distribution is not expected. Despite these differences, the measured spectrum still qualitatively aligns with the theoretical prediction, confirming the validity of the FM signal generation and analysis.

e. Differentiate the FM signal and plot the resulting signal and verify the frequency"

1. Code Implementation

```
%% Q3-e: Differentiation and Envelope Analysis
dx_FM = gradient(x_FM, t);
env = abs(hilbert(dx_FM));

figure(5);
subplot(2,1,1);
plot(t*1e3, dx_FM);
xlabel('Time (ms)'); ylabel('Amplitude');
title('Q3-e: Differentiated FM Signal'); grid on;

subplot(2,1,2);
plot(t*1e3, env);
xlabel('Time (ms)'); ylabel('Envelope');
title('Q3-e: Envelope of Differentiated Signal'); grid on;

% Verify fm frequency in envelope
[pxx, f_env] = pwelch(env-mean(env), [], [], [], Fs);
[~, idx] = max(pxx);
fprintf('Q3-e: Dominant envelope frequency: %.2f kHz (Expected: %.1f kHz)\n',...
        f_env(idx)/1e3, fm/1e3);
```

Fig.54 Waveform diagram of the differentiated FM signal $x_{FM}(t)$ and the code demonstration for frequency verification

2. Explanation of the Code

First, it uses the ***gradient*** function to differentiate the FM signal ***x_FM*** with respect to time ***t***, obtaining ***dx_FM*** to reflect the instantaneous change in the signal's frequency. Then, it applies the `hilbert` function to ***dx_FM***, takes the absolute value, and gets the signal envelope ***env***. Next, it creates a new figure window and divides it into two sub - plots, where the waveforms of ***dx_FM*** and ***env*** over time (converted to milliseconds) are plotted respectively, with corresponding titles and axis labels added. Finally, it uses the ***pwelch*** function to calculate the power spectral density of the envelope signal after removing the DC component, finds the frequency corresponding to the maximum power, outputs it as the dominant frequency of the envelope, and compares it with the expected modulation frequency to verify whether the envelope contains the expected modulation frequency component.

3. Result presentation

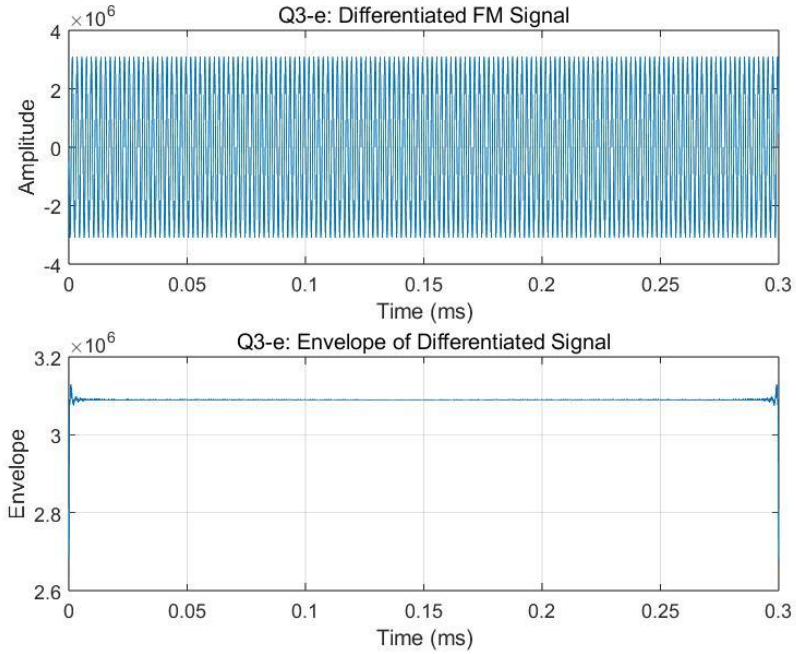


Fig.55 The result diagram of FM signal differentiation and its envelope analysis

4. Result analysis

- **Upper part "Q3 - e: Differentiated FM Signal"**: It presents the waveform of the differentiated FM signal. You can observe that the signal exhibits high - frequency oscillations. This is because after differentiating the FM signal, the frequency changes of the signal are highlighted. Since the frequency of the FM signal varies with the modulating signal, the differentiation operation makes these frequency changes manifest as high - frequency oscillations in the time domain. The amplitude fluctuates between -4×10^6 and 4×10^6, reflecting the rate and extent of the frequency change of the FM signal.
- **Lower part "Q3 - e: Envelope of Differentiated Signal"**: It shows the envelope of the differentiated signal. The envelope line is relatively stable, fluctuating around 3×10^6. This indicates that although the differentiated signal has intense high - frequency oscillations, the envelope remains relatively stable overall. The envelope reflects the changing trend of the signal amplitude. Its stability suggests that in this FM signal, although the frequency is constantly changing, the amplitude change follows a certain pattern and does not show significant fluctuations.

f. Apply a hard limiter to the FM signal and plot the output signal

1. Code Implementation

```
%% Q3-f: Hard Limiter
threshold = 0.1;
x_limited = min(max(x_FM, -threshold), threshold);

figure(6);
plot(t*1e3, x_limited);
xlabel('Time (ms)'); ylabel('Amplitude');
title('Q3-f: Hard Limited FM Signal'); grid on;
ylim([-threshold*1.1, threshold*1.1]);
```

Fig.56 Output waveform diagram of the FM signal processed by the hard limiter and its MATLAB code implementation

2. Explanation of the Code

First, a limiting threshold of 0.1 is set. Then, the ***min*** and ***max*** functions are used to limit the amplitude of the FM signal x_FM between -0.1 and 0.1, resulting in the limited signal $x_limited$. Subsequently, a new figure window is created to plot the waveform of the limited signal against time (converted to milliseconds). Axis labels and a title are added, the grid is turned on, and the display range of the y - axis is set to fully show the amplitude of the limited signal.

3. Result presentation

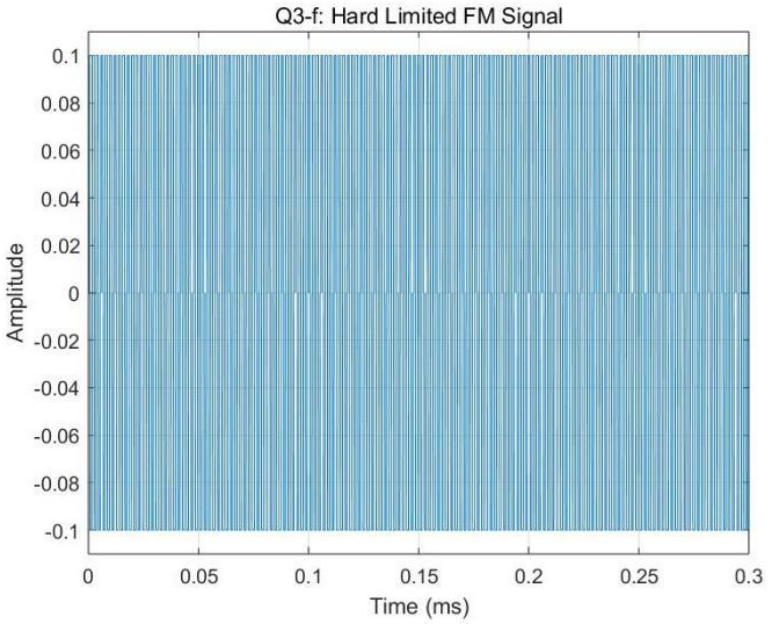


Fig.57 Time-domain Waveform of Hard-limited FM Signal

4. Result analysis

This figure corresponds to Q3 - f and shows the time-domain waveform of the FM signal after hard limiting. As can be seen from the figure, the amplitude of the signal is strictly limited between -0.1 and 0.1, which is consistent with the limiting threshold of 0.1 set in the code. The hard limiting process truncates the parts of the original FM signal that exceed this range, preventing the signal amplitude from going beyond the set limits. The signal continues to oscillate along the time axis, indicating that despite the amplitude limitation, the frequency modulation characteristic of the FM signal still exists, meaning the signal is still undergoing frequency changes. Through this hard limiting process, abnormal amplitude values that are too high or too low in the signal can be removed. In some communication scenarios, it helps to stabilize the signal amplitude and reduce non-linear distortion caused by excessive signal amplitude, while retaining the information carried by frequency modulation.

g. Obtain the frequency spectrum of the output signal of the hard limiter and compare and analyze it with the original frequency spectrum

1. Code Implementation

```
%% Q3-g: Hard Limiter Spectrum
X_limited = fftshift(fft(x_limited)/N);

figure(7);
subplot(2,1,1);
plot(f/1e3, 20*log10(abs(X_limited)));
xlabel('Frequency (kHz)'); ylabel('Magnitude (dB)');
title('Q3-g: Limited Signal Spectrum (Full)'); grid on;
xlim([-Fs/2/1e3, Fs/2/1e3]);

subplot(2,1,2);
plot(f/1e3, 20*log10(abs(X_limited)));
xlabel('Frequency (kHz)'); ylabel('Magnitude (dB)');
title('Q3-g: Limited Signal Spectrum (Zoomed)'); grid on;
xlim([fc/1e3-200, fc/1e3+200]); ylim([-100, 0]);
```

Fig.58 Comparison diagram between the frequency spectrum of the output signal of the hard limiter and the original frequency spectrum of the FM signal and its code demonstration

2. Explanation of the Code

First, it conducts a Fast Fourier Transform (FFT) on the hard-limited signal $x_limited$, uses $fftshift$ to move the zero frequency to the center of the spectrum, and then divides by the signal length N to obtain the normalized spectrum $X_limited$. Then, it creates a new figure window and divides it into two subplots. One subplot plots the entire spectrum, with the frequency (in kHz) on the x - axis and the logarithmic magnitude (in dB) of the spectrum on the y - axis. It adds labels and a title and sets the x - axis range to display the full spectrum. The other subplot zooms in on the spectrum near the carrier frequency, setting the x and y - axis ranges to facilitate the observation of the spectral details of the carrier and its sidebands.

3. Result presentation

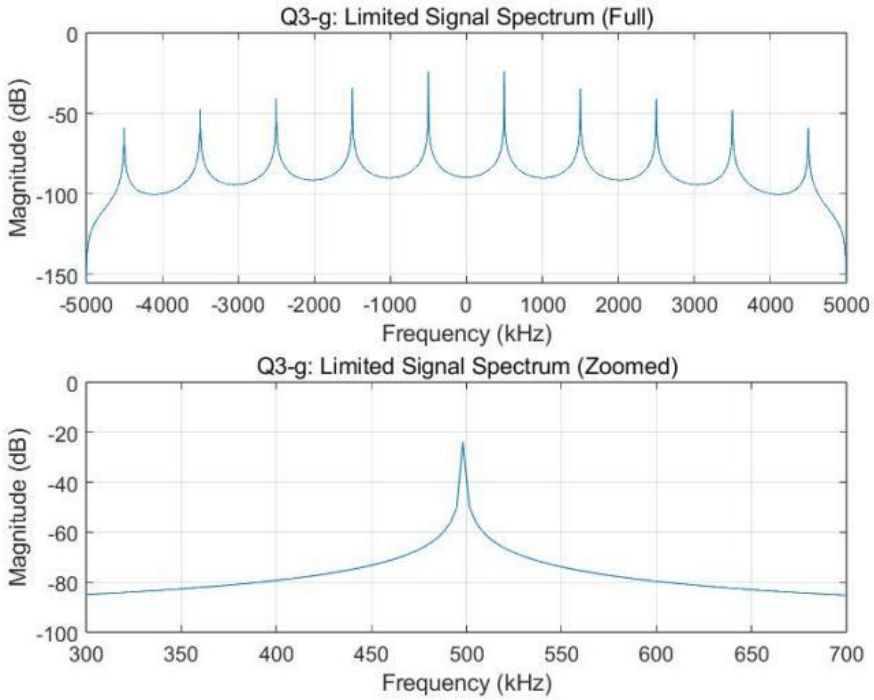


Fig.59 Spectra of Hard - Limited FM Signal (Full View and Zoomed - in

4. Result analysis

- **Upper part "Q3 - g: Limited Signal Spectrum (Full)"**: It presents the full - spectrum plot of the hard - limited FM signal. The spectrum is symmetrically distributed around the zero frequency, which is consistent with the characteristics of the spectrum of a real - valued signal. There are multiple peaks in the spectrum, and these peaks correspond to the carrier frequency of the FM signal and its sideband frequencies. The sidebands are evenly spaced around the carrier frequency, and the spacing is related to the modulation frequency. By observing the entire spectrum, one can understand the distribution range of the frequency components of the signal after hard - limiting processing and the approximate amplitude of each sideband, reflecting the overall spectral structure of the signal.
- **Lower part "Q3 - g: Limited Signal Spectrum (Zoomed)"**: It is a zoom - in view of the spectrum near the carrier frequency. In the figure, there is a distinct peak at the carrier frequency, which is the core frequency component of the FM signal. On both sides of the carrier frequency, the distribution of sidebands can be seen. Although the sideband details are more obvious in the zoom - in plot, the amplitude of the sidebands is relatively lower than that of the peak at the carrier frequency.

This plot is convenient for observing the amplitude and frequency details of the carrier and its nearby sidebands, which helps to analyze the specific effects of hard - limiting processing on the spectral characteristics of the carrier and sidebands, such as changes in sideband amplitude, and thus evaluate the changes in the signal in the frequency domain due to hard - limiting processing.

5. Comparison

Figure in Question g shows the spectrum of the FM signal after hard limiting, while the figure in Question c presents the spectrum of the original FM signal. There are significant differences between them. In terms of amplitude characteristics, as a non - linear operation, hard - limiting processing changes the amplitude relationship of each frequency component in the signal spectrum, making the sideband amplitude distribution complex, potentially suppressing or enhancing the amplitude of certain sidebands. In the spectrum of the original FM signal, the amplitude peak at the carrier frequency is obvious, and the sideband amplitude decays smoothly. Regarding frequency components, the non - linearity of hard limiting introduces new frequency components such as harmonics, complicating the spectral structure, whereas the spectrum of the original FM signal only contains the carrier and regularly distributed sidebands generated based on normal modulation. In terms of spectral shape, due to amplitude limitation and new frequency components, the spectrum of the hard - limited signal is no longer smooth, with irregular fluctuations. The spectrum of the original FM signal, however, has the carrier as the center, with the sideband amplitude decaying smoothly and presenting a regular shape.

h. Filter the output signal of the hard limiter to recover the FM signal and verify it

1. Code Implementation

```
%% Q3-h: Bandpass Filter Reconstruction
bpFilt = designfilt('bandpassfir', ...
                    'CutoffFrequency1', fc-2*(beta+1)*fm, ...
                    'CutoffFrequency2', fc+2*(beta+1)*fm, ...
                    'SampleRate', Fs, ...
                    'FilterOrder', 200);
x_filtered = filtfilt(bpFilt, x_limited);

X_filtered = fftshift(fft(x_filtered)/N);

figure(8);
subplot(2,1,1);
plot(t*1e3, x_filtered);
xlabel('Time (ms)'); ylabel('Amplitude');
title('Q3-h: Reconstructed FM Signal'); grid on;

subplot(2,1,2);
plot(f/1e3, 20*log10(abs(X_filtered)));
xlabel('Frequency (kHz)'); ylabel('Magnitude (dB)');
title('Q3-h: Reconstructed Spectrum'); grid on;
xlim([fc/1e3-150, fc/1e3+150]);
```

Fig.60 Frequency spectrum verification diagram of the FM signal recovered by band-pass filtering and its MATLAB code implementation

2. Explanation of the Code

First, it uses the designfilt function to design a 200th-order FIR band-pass filter. The upper and lower cutoff frequencies are calculated based on the carrier frequency, modulation index, and modulation frequency, and the sampling rate matches that of the signal. Then, it applies the filtfilt function to perform zero-phase bidirectional filtering on the hard-limited signal to obtain the filtered signal. Next, it conducts FFT on the filtered signal and normalizes it to get the spectrum. Finally, it creates a new figure window with two subplots. One subplot plots the time-domain waveform of the filtered signal, and the other plots the spectrum near the carrier frequency. Necessary labels and titles are added, and the grid is turned on.

3. Result presentation

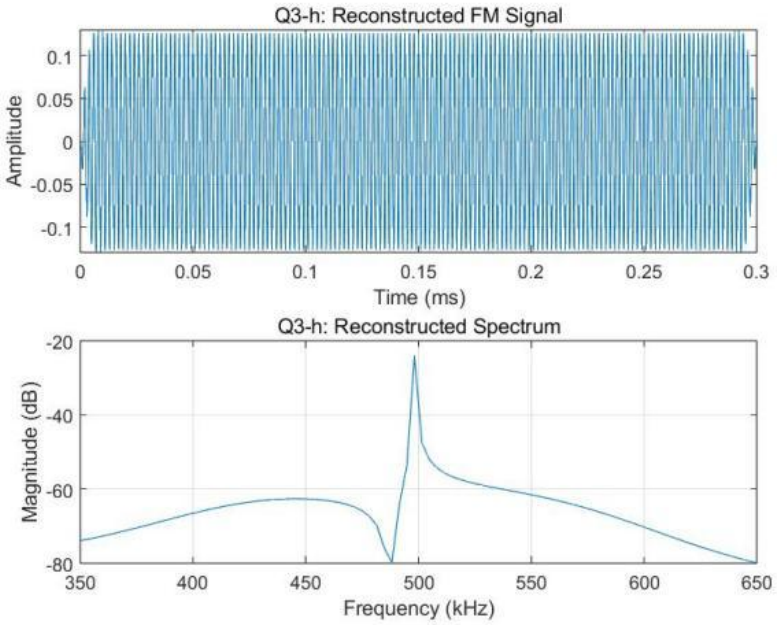


Fig.61 Reconstructed FM Signal and Its Time - Domain Spectrum Diagram

4. Result analysis

- **Time - domain plot (upper part)**: It shows the time - domain waveform of the reconstructed FM signal after being processed by the band - pass filter. The signal exhibits high - frequency oscillation characteristics, with the oscillation amplitude fluctuating between -0.1 and 0.1. This indicates that the band - pass filter has retained the amplitude characteristics of the signal to a certain extent. The periodic oscillation of the signal reflects the frequency - modulation characteristic of the FM signal, suggesting that the filter has successfully preserved the time - domain information related to frequency modulation. Despite the hard - limiting and filtering steps, the signal still maintains the typical time - domain form of an FM signal.
- **Frequency - domain plot (lower part)**: It presents the spectrum of the reconstructed FM signal. In the spectrum plot, a significant peak can be clearly observed at a certain frequency (presumably the carrier frequency, around 500 kHz), which corresponds to the carrier component of the FM signal. The spectrum amplitudes on both sides of the carrier frequency are relatively low and change smoothly. This indicates that the band - pass filter has effectively filtered out most of the non - target frequency components, retaining only the main frequency components related to the FM signal, highlighting the carrier and its nearby sideband information, and suppressing the spurious frequency components that

may be introduced by the hard - limiting process, making the reconstructed spectrum closer to the spectral characteristics of the original FM signal.

In terms of the time - domain waveform, the figure in the result graph shows high - frequency oscillations, which is consistent with the time - domain characteristics of an FM signal. This indicates that the band - pass filter has retained the frequency - modulation characteristics of the signal. From the frequency - domain perspective, in the "Q3 - h: Reconstructed Spectrum" graph, there is a distinct peak at the carrier frequency, and it effectively suppresses spurious frequency components while retaining the main frequency components related to the FM signal, which is in line with the spectral characteristics of the original FM signal. Considering both the time - domain and frequency - domain performances, the signal processed by the band - pass filter exhibits characteristics similar to the original FM signal $x_{FM}(t)$ in both the time - domain and frequency - domain. Therefore, the results meet the requirements of the question.